

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

CÁTEDRA DE ELECTRÓNICA DIGITAL III

TRABAJO PRÁCTICO INTEGRADOR

**“SISTEMA DE CONTROL DE ILUMINANCIA EN UN
RECINTO”**

Grupo N° 9

Alumnos:

Alasia, Santiago

Manzano, Maximo

Profesor:

Migliore, Emiliano

Comisión N° 3.3:

19/11/2025

TABLA DE CONTENIDO

- 1. PROYECTO**
- 2. DESCRIPCIÓN DEL SEP**
- 3. DESARROLLO DE HARDWARE**
 - 3.1 CIRCUITO DEL SSEP MICROCONTROLADOR**
 - 3.2 CIRCUITO DEL SSE OPT101**
 - 3.3 CIRCUITO DEL SSE DE COMUNICACIÓN DE DATOS**
 - 3.4 CIRCUITO DEL SSE ACOPLAMIENTO GALVÁNICO**
 - 3.5 CIRCUITO DEL SSE DRIVER / LÁMPARA LED**
 - 3.6 CIRCUITO DEL SSE DE VISUALIZACIÓN DE DATOS**
- 4. DESARROLLO DE FIRMWARE**
 - 4.1 FIRMWARE DEL SSE DE OPT101**
 - 4.2 FIRMWARE DEL SSE DE COMUNICACIÓN DE DATOS**
 - 4.3 FIRMWARE DEL SSE DRIVER / LÁMPARA LED**
 - 4.4 FIRMWARE DEL SSE DE VISUALIZACIÓN DE DATOS**
- 5. INTERFAZ GRÁFICA**
- 6. PRUEBAS DE SISTEMA**
- 7. CONCLUSIONES**
- 8. BIBLIOGRAFÍA Y REFERENCIAS**
- 9. ANEXO**
 - 8.1 CIRCUITO ESQUEMÁTICO DEL SEP**
 - 8.2 FIRMWARE DEL SEP**
 - 8.3 LISTA DE MATERIALES**
 - 8.4 HOJAS DE DATOS**

TÉRMINOS Y DEFINICIONES

TPI: Trabajo Práctico Integrador

SEP: Sistema Electrónico Programable

SSE: Subsistema Electrónico

SSEP: Subsistema Electrónico Programable

1. PROYECTO

El presente trabajo integrador se enfoca en el diseño y en la implementación de un “Sistema de Control de Iluminancia” para un recinto. El proyecto tiene como objetivo principal establecer y mantener un nivel de iluminancia. El mismo se basa en un SSEP LPC1769 como sistema microcontrolador. Este dispositivo es el encargado de adquirir y procesar la señal de la variable de control a través del sensor OPT101, un fotodiodo monolítico con amplificador de transimpedancia que proporciona una lectura analógica precisa. Para la actuación, se utiliza una lámpara LED de 12V dimerizable, cuya intensidad se modula directamente mediante una señal de modulación por ancho de pulso (PWM) generada por el microcontrolador.

La interacción y el monitoreo del sistema se gestionan mediante una interfaz de usuario desarrollada en una PC a través de comunicación serial. Esta interfaz tiene como fin permitir al usuario establecer el Set-Point y ofrecer una visualización de las últimas mediciones.

Además, se incluyen 4 displays de 7 segmentos para facilitar la visualización del Set-Point en el equipo.

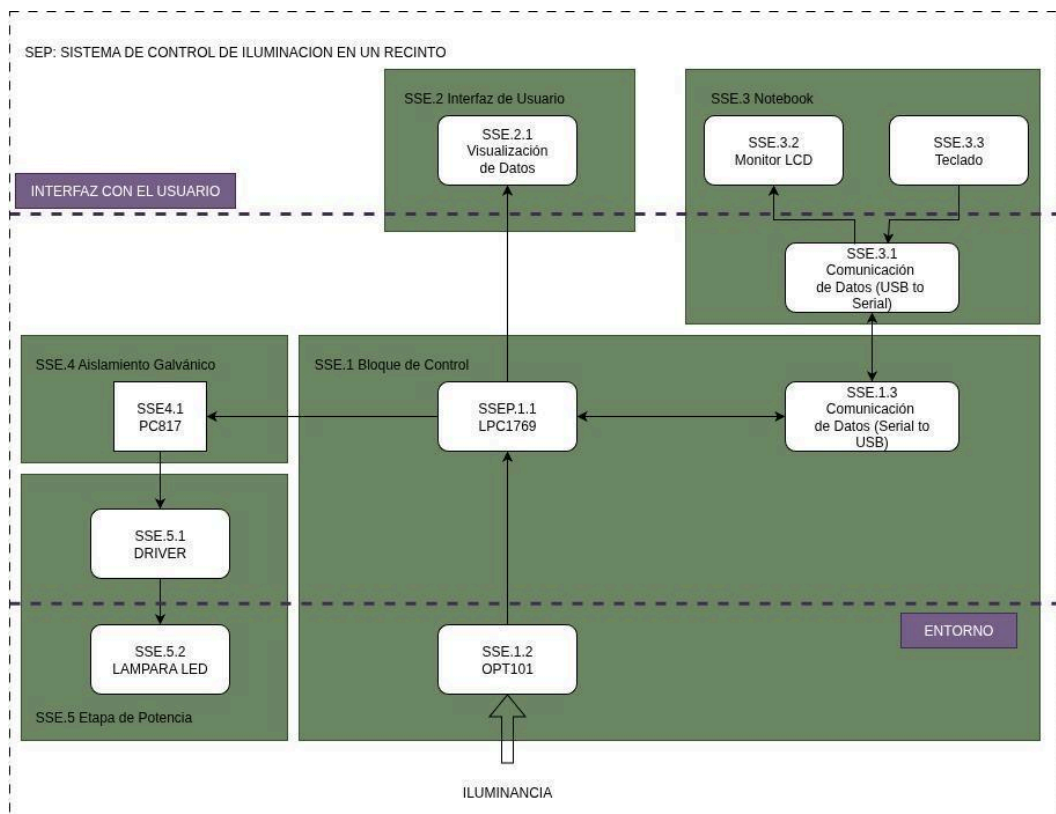


Figura 1: Diagrama de Bloques del SEP

2. DESCRIPCIÓN DEL SEP

El usuario puede interactuar con el sistema de dos maneras. La primera es mediante una interfaz gráfica (SSE.3), en la cual el usuario podría ver representado en una gráfica, las últimas mediciones de iluminancia y contará con la posibilidad de ajustar el valor deseado de iluminancia. La segunda forma es a través de unos displays 7 segmentos (SSE.2), que tienen como objetivo mostrar el valor de iluminancia establecido por el usuario.

El sistema mide iluminancia, utilizando el sensor OPT101, una vez tomada esa medición, se ajustará la intensidad de la lámpara LED, utilizando un PWM, para así lograr ajustar el valor de iluminancia establecido anteriormente por el usuario. Para realizar el control de la intensidad de la lámpara LED, se utiliza un driver (SSE.5.1) y una etapa de aislamiento (SSE.4)

Para calcular el valor de la resistencia del “Led ON/OFF” se hizo uso de la ecuación de Kirchhoff la cual podemos plantear como:

$$V = I * R + V_D$$

Donde V_D representa la caída de tensión del diodo led, esta depende de cada color de diodo y sale de la siguiente gráfica:

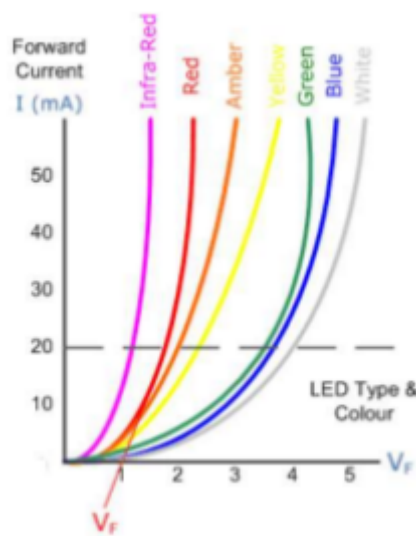


Figura 4: Curvas de caída de tensión vs corriente para diodos leds.

Entonces, nuestra fórmula quedaría de la siguiente manera:

$$V_R = 10mA * R + 1,2 v$$

Despejando el valor de interés y reemplazando los demás valores, tenemos;

$$\frac{5v - 1,2v}{10mA} = R \Rightarrow R = 380\Omega$$

Este valor es teórico, en la práctica, se utilizó valores de resistencia de 330Ω.

3.2 CIRCUITO DEL SSE OPT

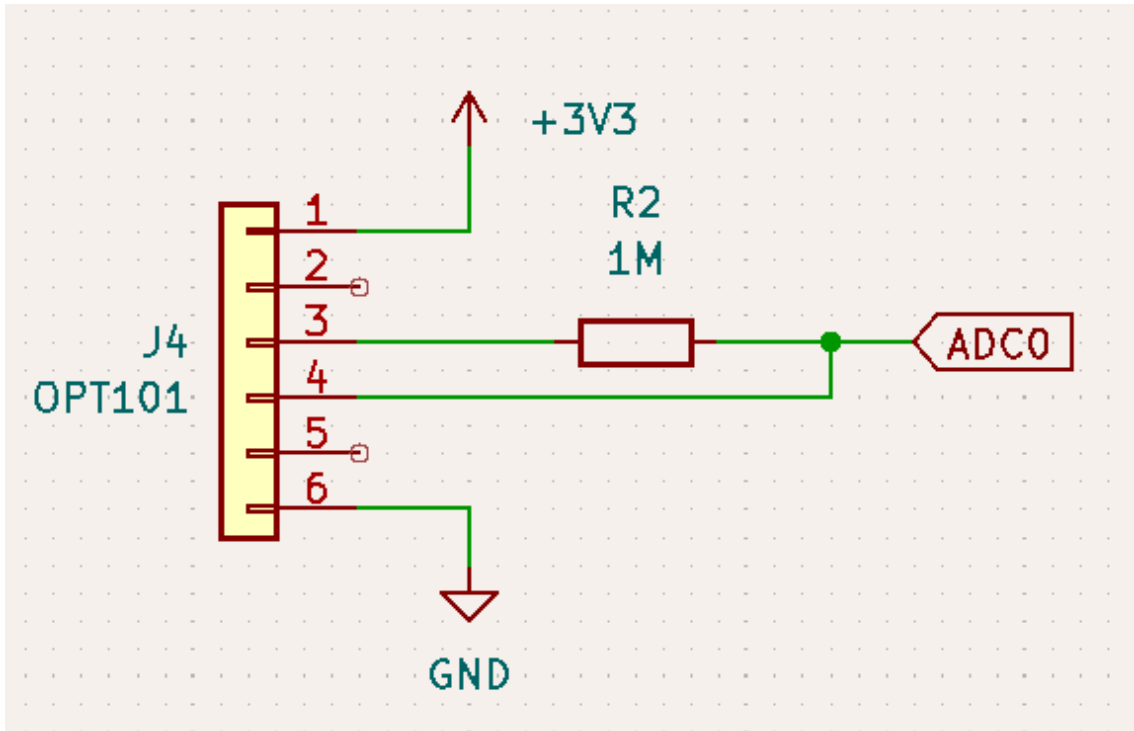


Figura 5: Circuito Esquemático SSE 1.2

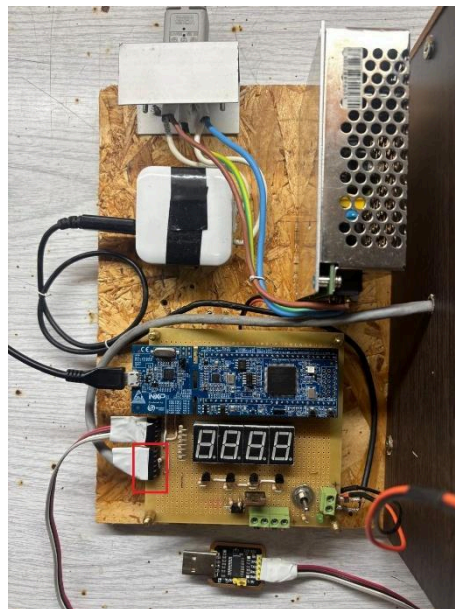


Figura 6: Imagen del SSE 1.2 en la placa

En este caso, simplemente se utilizó un conector hembra de 6 pines para el conexionado. Siguiendo la hoja de datos, se agregó una resistencia de $1M\Omega$ entre la salida y el cuarto pin del sensor.

3.3 CIRCUITO DEL SSE DE COMUNICACIÓN DE DATOS

Como se puede observar en la *Figura 7*, para utilizar este módulo simplemente debemos conectar los pines de comunicación serial de la LPC1769 a los pines del módulo. En la *Figura 9* podemos observar más en detalle los pines de este módulo.

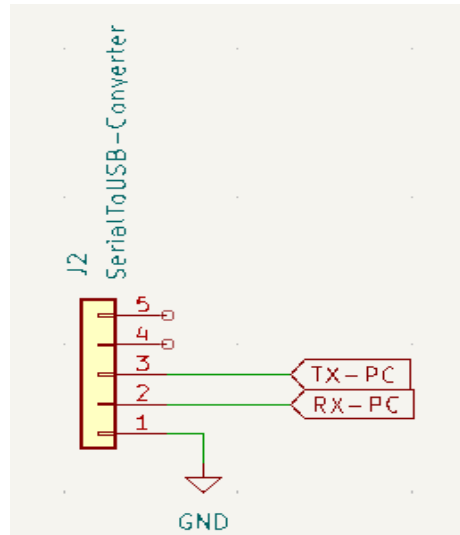


Figura 7: Circuito Esquemático SSE 1.3

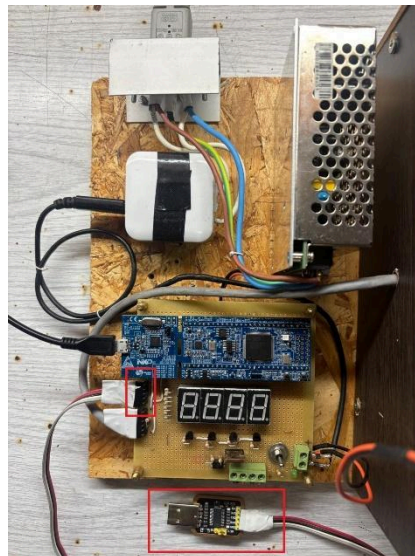


Figura 8: Imagen del SSE 1.3 en la placa



Figura 9: Módulo TTL-USB

3.4 CIRCUITO DEL SSE ACOPLAMIENTO GALVÁNICO

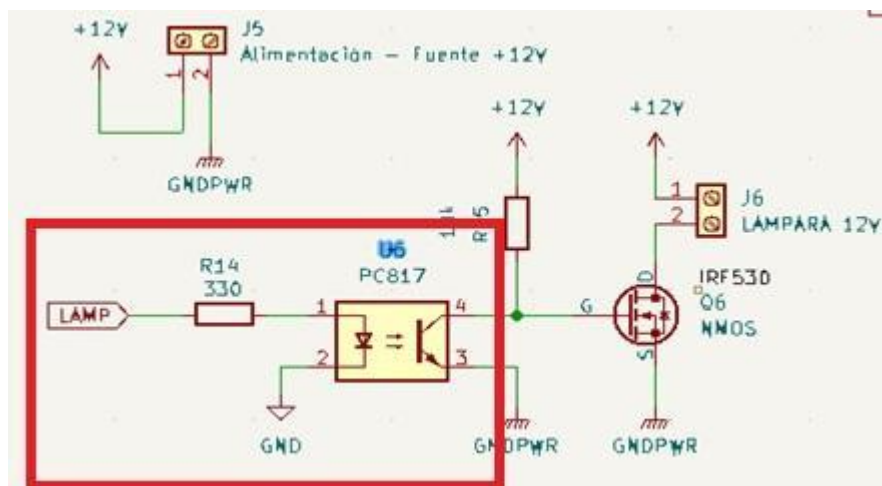


Figura 10: Circuito Esquemático SSE 4.1

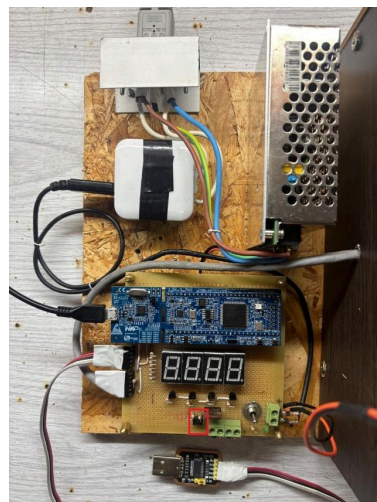
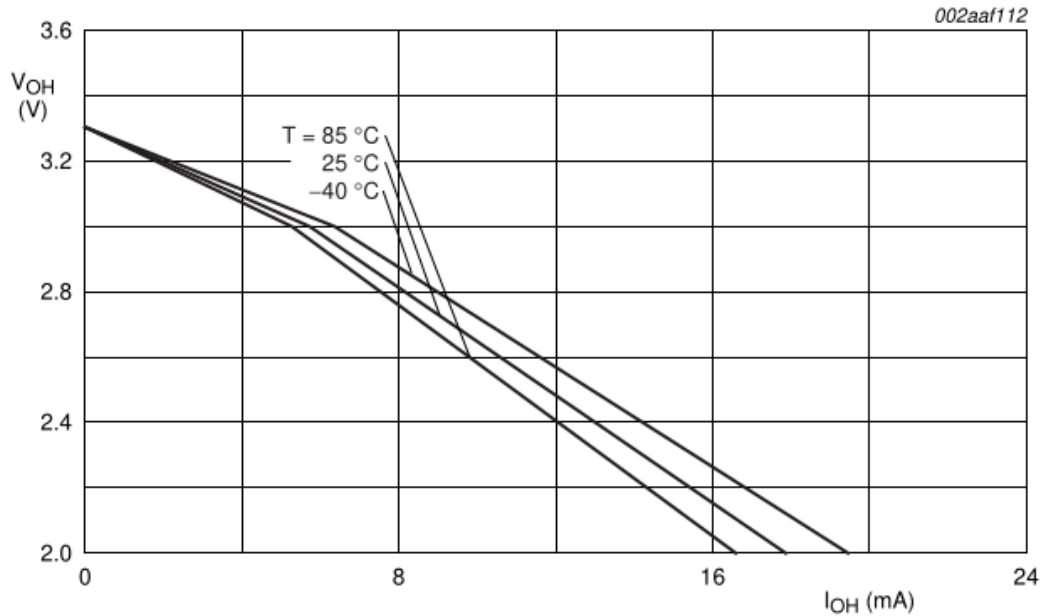


Figura 11: Imagen del Circuito SSE 4.1 en la placa

Para el cálculo de la resistencia de este SSE se siguieron las mismas fórmulas que en el SSEP MICROCONTROLADOR. La única diferencia es que en este caso la tensión será de 3.1v debido a que se trata de un pin de propósito general de la placa. En la siguiente gráfica obtenida de la hoja de datos de la placa podemos observar lo anteriormente mencionado.



Conditions: $V_{DD(REG)(3V3)} = V_{DD(3V3)} = 3.3\text{ V}$; standard port pins.

Por lo tanto, la fórmula quedaría:

$$\frac{3.1\text{v} - 1.2\text{v}}{5\text{mA}} = R \Rightarrow R = 300\Omega \approx 330\Omega$$

3.5 CIRCUITO DEL SSE DRIVER / LÁMPARA LED

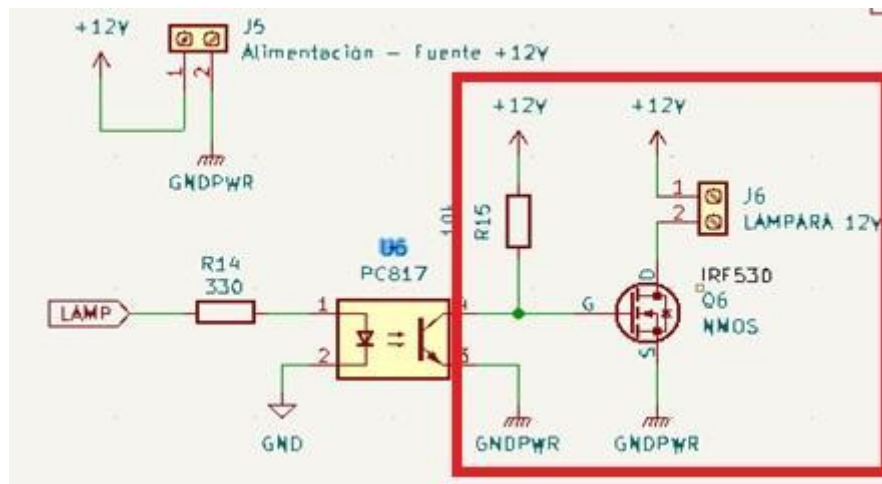


Figura 12: Circuito Esquemático SSE 5.1 y 5.2

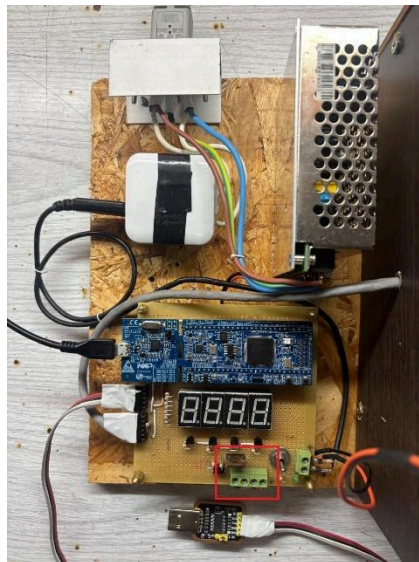


Figura 13: Imagen del Circuito SSE 5.1 y 5.2 en la placa

Para los cálculos de este circuito se aplicó la ley de Kirchhoff de las tensiones sobre la rama de la carga y se obtuvo el siguiente resultados:

$$V_{cc} - I_d * R_{carga} - V_D - V_{ds} = 0$$

Donde, R_{carga} es la resistencia de la lámpara y V_D es la caída de tensión de los leds internos.

Si reemplazamos con los valores, tenemos:

$$12v - I_d * 23.5\Omega - 9v - V_{ds} = 0$$

En nuestro caso queremos usarlo en corte y saturación, por lo tanto I_d y V_{ds} quedarían de la siguiente manera:

1. Para saturación:

$$I_d = \frac{V_{cc} - V_D}{R_{carga}} \quad V_{ds} = 0v$$

2. Para corte:

$$I_d = 0A \quad V_{ds} = V_{cc}$$

3.6 CIRCUITO DEL SSE DE VISUALIZACIÓN DE DATOS

En este SSE se utilizaron displays cátodo común y para la multiplexación de estos, transistores BC547.

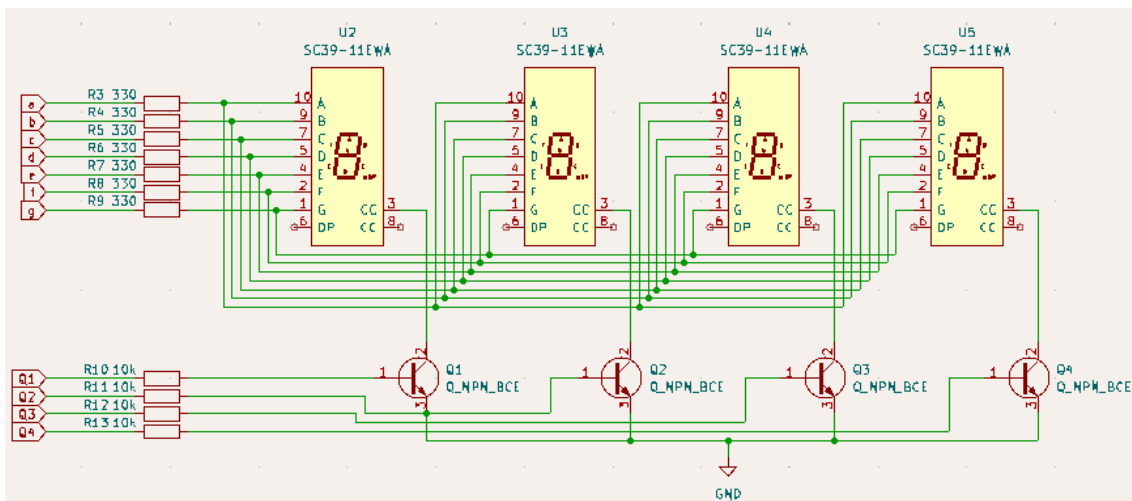


Figura 14: Circuito Esquemático SSE 2.1

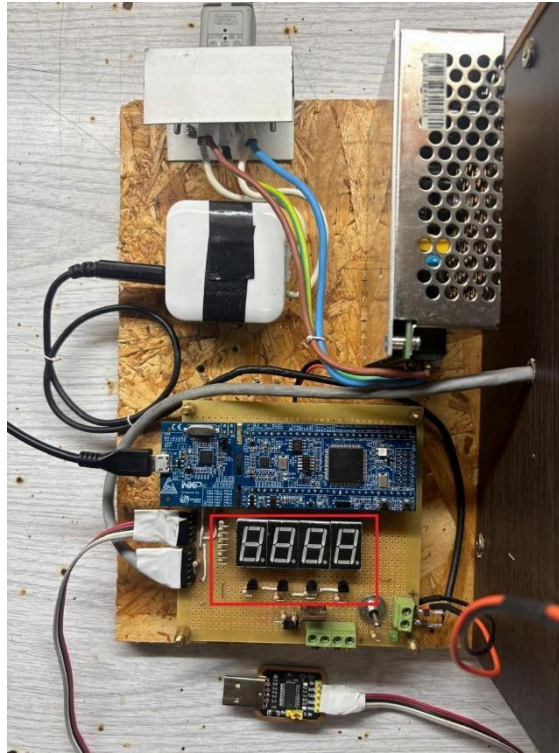


Figura 15: Imagen del Circuito SSE 2.1 en la placa

Para el cálculo de los valores de las resistencias de cada segmento del display y para los transistores se plantearon las mallas de entrada.

$$V - V_{SS} = I_{RD\#} * R_{D\#} + V_D + V_{ce(sat)}$$

$$V - V_{SS} = R_{B\#} * I_{RB\#} + V_{be(sat)}$$

Los valores de $V_{ce(sat)}$ y de $V_{be(sat)}$ los buscamos en el datasheet.

Collector–Emitter Saturation Voltage ($I_C = 10\text{ mA}$, $I_B = 0.5\text{ mA}$) ($I_C = 100\text{ mA}$, $I_B = 5.0\text{ mA}$) ($I_C = 10\text{ mA}$, $I_B = \text{See Note 1}$)	$V_{CE(sat)}$	—	0.09	0.25	V
		—	0.2	0.6	
		—	0.3	0.6	
Base–Emitter Saturation Voltage ($I_C = 10\text{ mA}$, $I_B = 0.5\text{ mA}$)	$V_{BE(sat)}$	—	0.7	—	V

Como $V_{ce(sat)}$ es demasiado chica, la consideramos como que su valor es de 0v.

Una vez planteadas las mallas, procedemos a realizar el despeje necesario para el cálculo de las resistencias.

$$R_{D\#} = \frac{V - V_{SS} - V_{ce(sat)} - V_D}{I_{RD\#}} = \frac{3,1V - 0V - 0V - 1,2V}{5mA} = 380\Omega \Rightarrow R_{RD\#} = 330\Omega$$

$$R_{B\#} = \frac{V - V_{SS} - V_{be(sat)}}{I_{RB\#}}$$

Para el cálculo de esta malla, tenemos que calcular $I_{RB\#}$ que se hace con la siguiente fórmula:

$$I_{RB\#} = \frac{I_c}{hfe} = \frac{Id * N^{\circ} \text{ de segmentos}}{hfe}$$

El valor de hfe lo encontramos en la hoja de datos del transistor:

DC Current Gain		h_{FE}				
(I _C = 10 μ A, V _{CE} = 5.0 V)	BC547A/548A	—	—	90	—	—
	BC546B/547B/548B	—	—	150	—	—
	BC548C	—	—	270	—	—
(I _C = 2.0 mA, V _{CE} = 5.0 V)	BC546	110	—	—	450	—
	BC547	110	—	—	800	—
	BC548	110	—	—	800	—
	BC547A/548A	110	180	—	220	—
	BC546B/547B/548B	200	290	—	450	—
	BC547C/BC548C	420	520	—	800	—
(I _C = 100 mA, V _{CE} = 5.0 V)	BC547A/548A	—	120	—	—	—
	BC546B/547B/548B	—	180	—	—	—
	BC548C	—	300	—	—	—

Para los cálculos de corriente vamos a usar un hfe de 110, por lo tanto la corriente sería la siguiente:

$$I_{RB\#} = \frac{I_c}{hfe} = \frac{5mA * 7}{110} = 0,32mA$$

Entonces el valor de $R_{B\#}$ es:

$$R_{B\#} = \frac{V - V_{SS} - V_{be(sat)}}{I_{RB\#}} = \frac{3,1v - 0v - 0,7v}{0,32mA} = 7500\Omega \Rightarrow R_{B\#} = 10k\Omega$$

4. DESARROLLO DE FIRMWARE

Para tener una mejor comprensión de cómo los periféricos interactúan entre ellos, se realizó el siguiente diagrama que trata de plasmarlo.

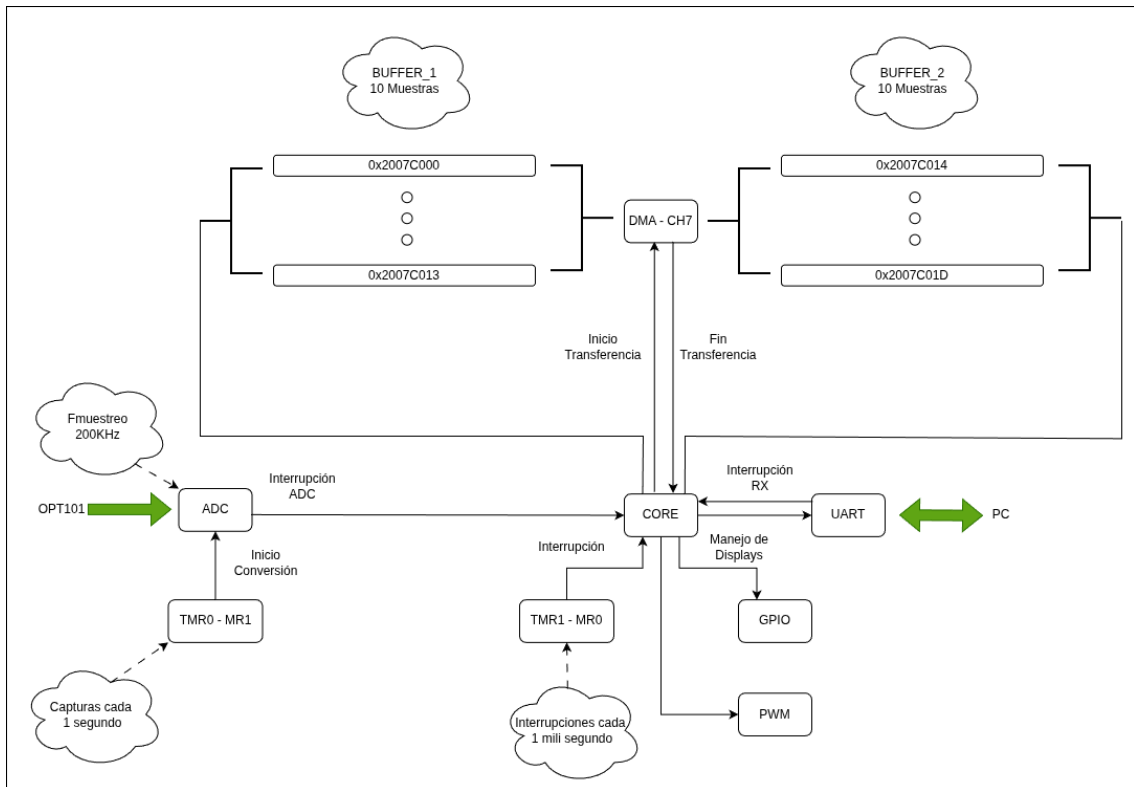


Figura 16: Diagrama de los Periféricos.

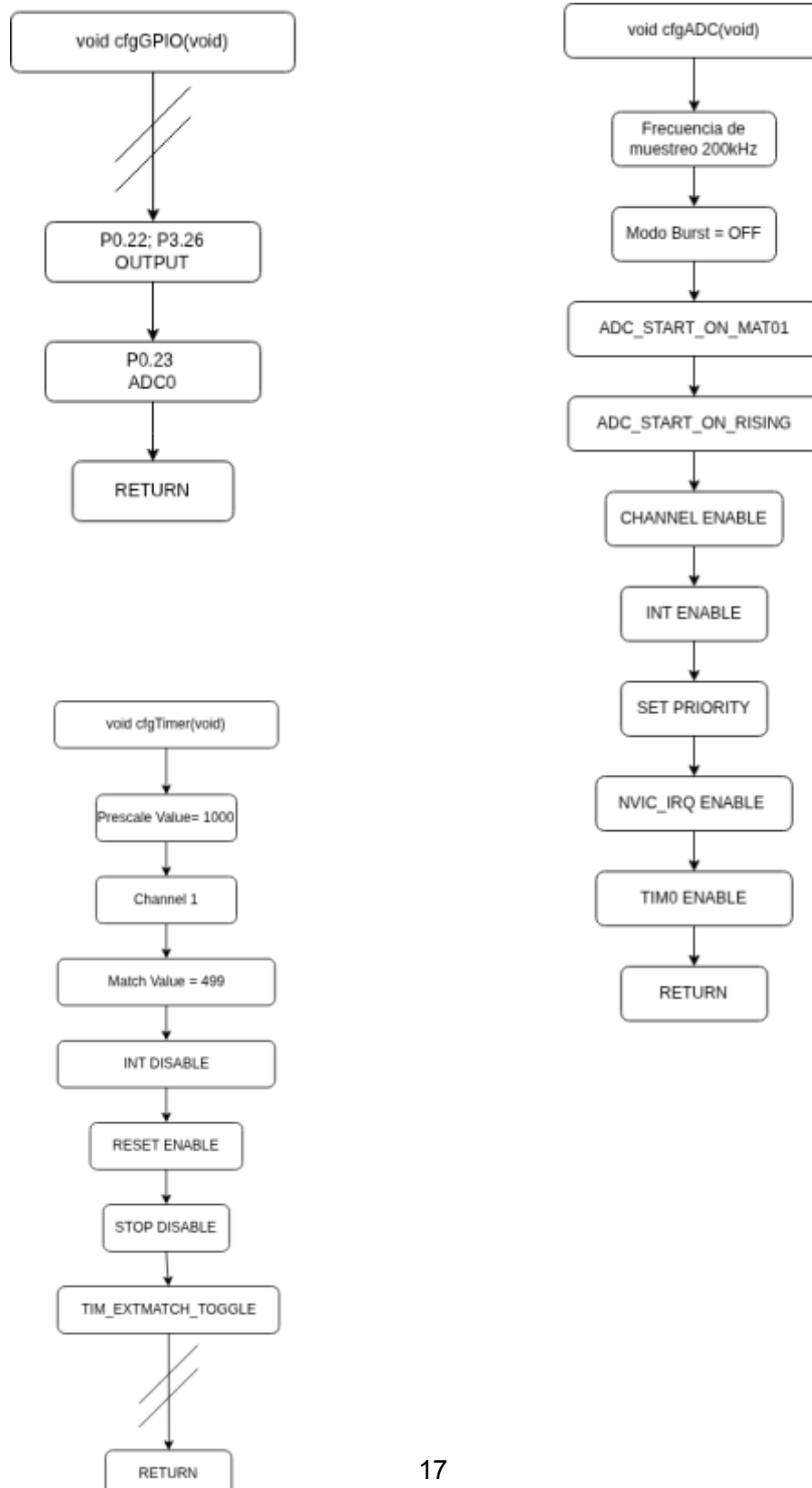
Gracias a la *Figura 16* se puede interpretar con mayor precisión la configuración del sistema y el manejo de los periféricos involucrados. El funcionamiento general del sistema es el siguiente:

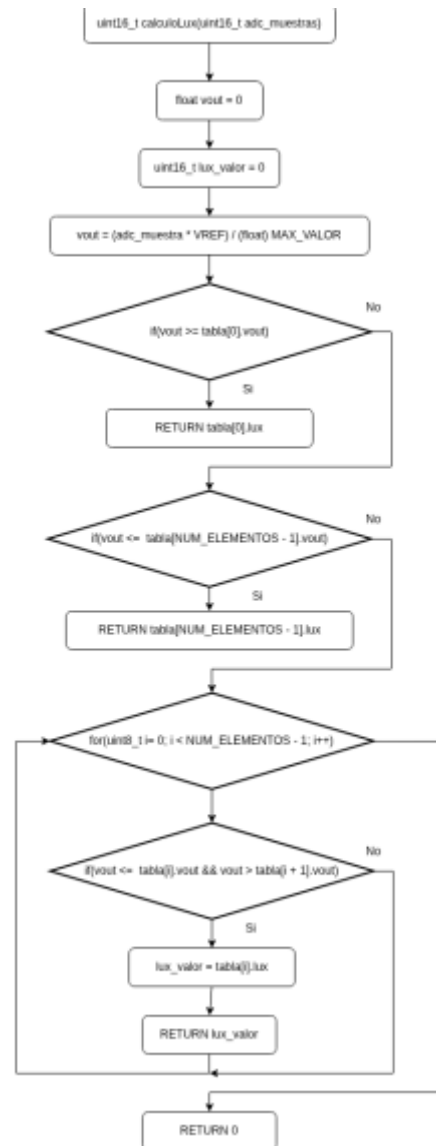
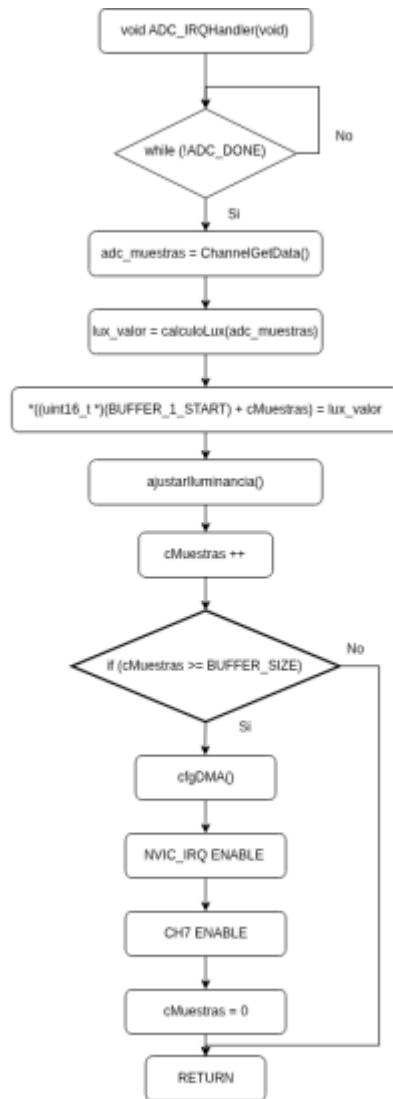
El Timer 0, configurado en modo *Match 1*, se encarga de disparar la conversión del ADC. Una vez finalizada dicha conversión, se genera la interrupción de fin de conversión, la cual es atendida por el *core*. En esa rutina se almacenan las muestras en el Buffer 1. Cuando este buffer se completa, se habilita la transferencia M2M mediante DMA.

Al finalizar la transferencia, el DMA genera una interrupción que nuevamente es atendida por el *core*, el cual se encarga de recorrer el Buffer 2 y enviar las muestras a través de la UART.

Además, se configuran otros periféricos, como el Timer 1 en modo *Match 0*, que se utiliza para el manejo del multiplexado de los displays o para la generación de la señal PWM, según corresponda.

4.1. FIRMWARE DEL SSE DE OPT101





```

void cfgGPIO(void) {
    /* Pin del Led Testigo */
    PINSEL_CFG_Type cfgPinLEDRed;
    PINSEL_CFG_Type cfgPinLEDBlue;

    // Configuración del LED en P0.22
    cfgPinLEDRed.Portnum = PINSEL_PORT_0;
    cfgPinLEDRed.Pinnum = PINSEL_PIN_22;
    cfgPinLEDRed.Funcnum = PINSEL_FUNC_0;
    cfgPinLEDRed.Pinmode = PINSEL_PINMODE_TRISTATE;
    cfgPinLEDRed.OpenDrain = PINSEL_PINMODE_NORMAL;

    // Configuración del LED en P3.26
    cfgPinLEDBlue.Portnum = PINSEL_PORT_3;
    cfgPinLEDBlue.Pinnum = PINSEL_PIN_26;
    cfgPinLEDBlue.Funcnum = PINSEL_FUNC_0;
    cfgPinLEDBlue.Pinmode = PINSEL_PINMODE_TRISTATE;
    cfgPinLEDBlue.OpenDrain = PINSEL_PINMODE_NORMAL;

    PINSEL_ConfigPin(&cfgPinLEDRed);
    PINSEL_ConfigPin(&cfgPinLEDBlue);

    GPIO_SetDir(PORT_0, PIN_22, OUTPUT);
    GPIO_SetValue(PORT_0, PIN_22);

    GPIO_SetDir(PORT_3, PIN_26, OUTPUT);
    GPIO_SetValue(PORT_3, PIN_26);

    /* Pin del ADC */
    PINSEL_CFG_Type cfgPinADC;

    // Configuración del pin P0.23 ADC Channel 0
    cfgPinADC.Portnum = PINSEL_PORT_0;
    cfgPinADC.Pinnum = PINSEL_PIN_23;
    cfgPinADC.Funcnum = PINSEL_FUNC_1;
    cfgPinADC.Pinmode = PINSEL_PINMODE_TRISTATE;
    cfgPinADC.OpenDrain = PINSEL_PINMODE_NORMAL;

    PINSEL_ConfigPin(&cfgPinADC);
}

```

```

void cfgADC(void){
    ADC_Init(LPC_ADC, 200000);

    ADC_BurstCmd(LPC_ADC, DISABLE);
    ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01);
    ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_RISING);
    ADC_ChannelCmd(LPC_ADC, 0, ENABLE);
    ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);

    NVIC_SetPriority(ADC_IRQn, 2);
    NVIC_EnableIRQ(ADC_IRQn);

    TIM_Cmd(LPC_TIM0, ENABLE);

    return;
}

```

```

void cfgTimer(void){
    /* Configuración del TMR0 en modo match1 - (TMR0MR1 cada 1s) */
    TIM_TIMERCFG_Type cfgTMR0Mode;
    TIM_MATCHCFG_Type cfgTMR0MAT1;

    // TMR0
    cfgTMR0Mode.PrescaleOption = TIM_PRESCALE_USVAL;
    cfgTMR0Mode.PrescaleValue = 1000; // 1 ms

    // MATCH 1
    cfgTMR0MAT1.MatchChannel = 1;
    cfgTMR0MAT1.MatchValue = 499;
    cfgTMR0MAT1.IntOnMatch = DISABLE;
    cfgTMR0MAT1.ResetOnMatch = ENABLE;
    cfgTMR0MAT1.StopOnMatch = DISABLE;
    cfgTMR0MAT1.ExtMatchOutputType = TIM_EXTMATCH_TOGGLE;

    TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTMR0Mode);
    TIM_ConfigMatch(LPC_TIM0, &cfgTMR0MAT1);

    return;
}

```

```

typedef struct{
    float vout;
    uint16_t lux;
    uint8_t dc;
} Elemento;

static const Elemento tabla[] = {
    {2.68, 295, 0},
    {2.58, 286, 5},
    {2.48, 258, 20},
    {2.38, 231, 35},
    {2.28, 203, 38},
    {2.18, 181, 42},
    {2.08, 168, 45},
    {1.98, 156, 48},
    {1.88, 144, 52},
    {1.78, 134, 55},
    {1.68, 124, 58},
    {1.58, 112, 63},
    {1.48, 99, 67},
    {1.38, 84, 75},
    {1.28, 68, 82},
    {1.18, 53, 84},
    {1.08, 42, 88},
    {0.98, 32, 92},
    {0.88, 19, 96},
    {0.78, 10, 98},
    {0.68, 9, 98},
    {0.58, 8, 98},
    {0.48, 7, 99},
    {0.38, 6, 99},
    {0.28, 5, 100},
    {0.18, 4, 100},
};

uint16_t calculoLux(uint16_t adc_muestra){
    float vout = 0;
    uint16_t lux_valor = 0;

    vout = (adc_muestra * VREF) / (float)MAX_VALOR;

    if (vout >= tabla[0].vout){
        return tabla[0].lux;
    }

    if (vout <= tabla[NUM_ELEMENTOS - 1].vout){
        return tabla[NUM_ELEMENTOS - 1].lux;
    }

    for (int i = 0; i < NUM_ELEMENTOS - 1; i++){
        if ( (vout <= tabla[i].vout) && (vout > tabla[i+1].vout) ){
            lux_valor = tabla[i].lux;
            return lux_valor;
        }
    }

    return 0;
}

```

```

void ADC_IRQHandler(void){
    static uint8_t cMuestras = 0;
    uint16_t adc_muestra = 0;
    uint16_t lux_valor = 0;

    while(!(ADC_ChannelGetStatus(LPC_ADC, ADC_CHANNEL_0, ADC_DATA_DONE)));

    adc_muestra = ADC_ChannelGetData(LPC_ADC, ADC_CHANNEL_0);
    lux_valor = calculoLux(adc_muestra);

    *((uint16_t *) (BUFFER_1_START) + cMuestras) = lux_valor;
    GPIO_ClearValue(PORT_0, PIN_22);

    ajustarIluminancia();

    cMuestras++;

    if(cMuestras >= BUFFER_SIZE){
        GPIO_SetValue(PORT_0, PIN_22);

        cfgDMA();

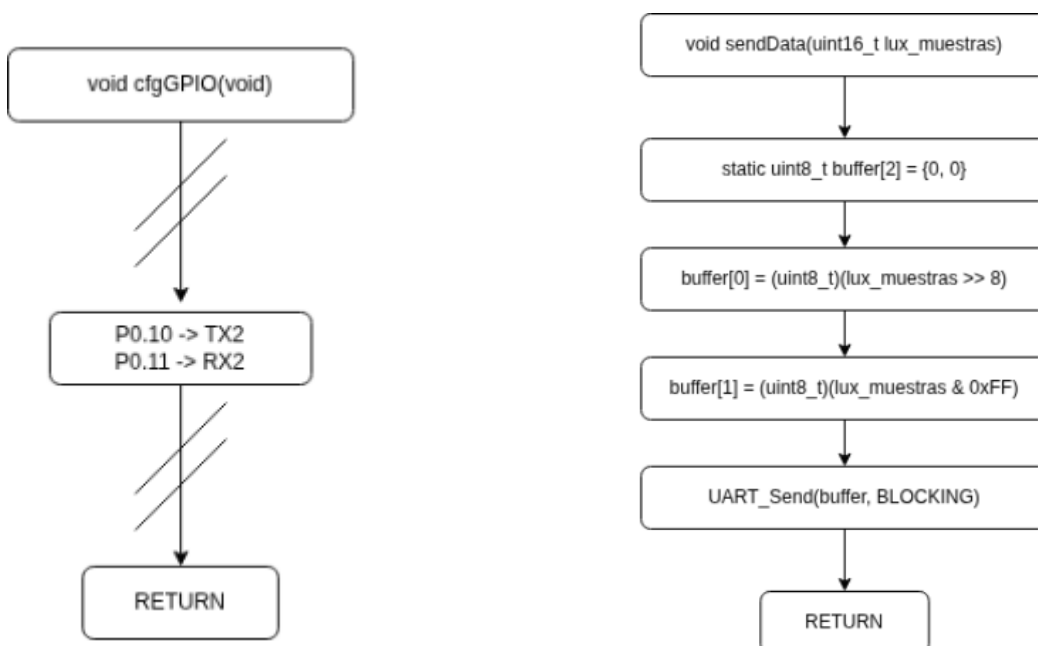
        NVIC_EnableIRQ(DMA_IRQn);
        GPDMA_ChannelCmd(7, ENABLE);

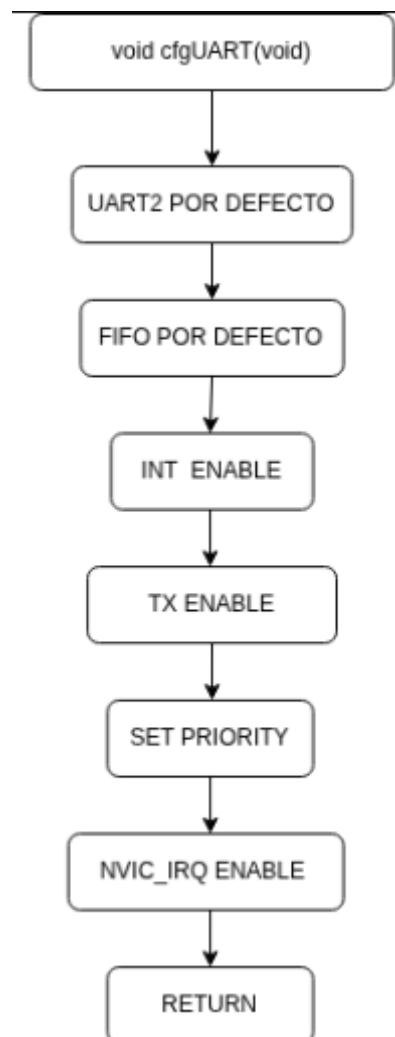
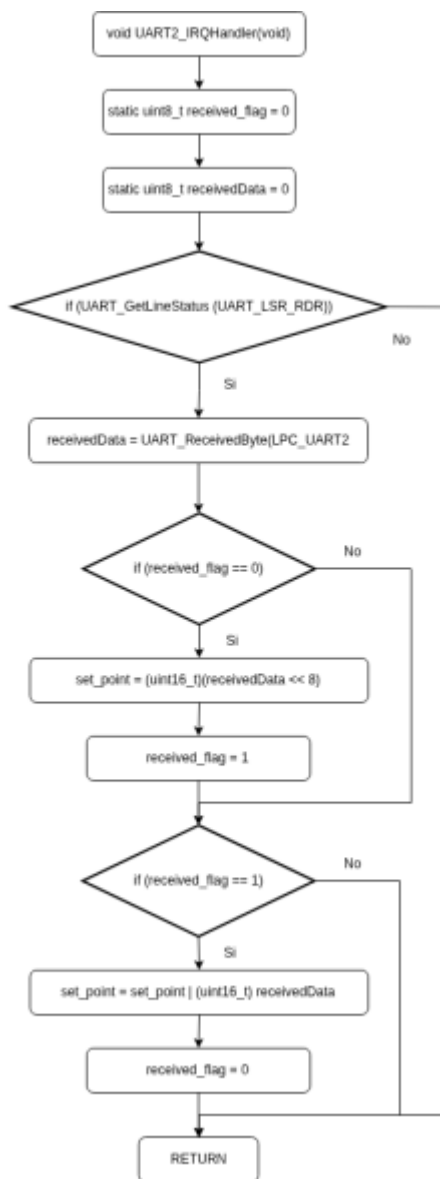
        cMuestras = 0;
    }

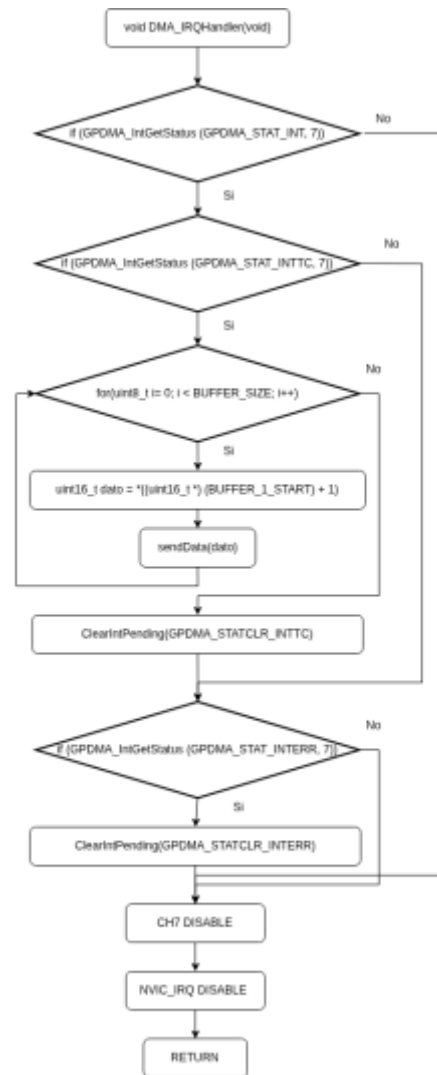
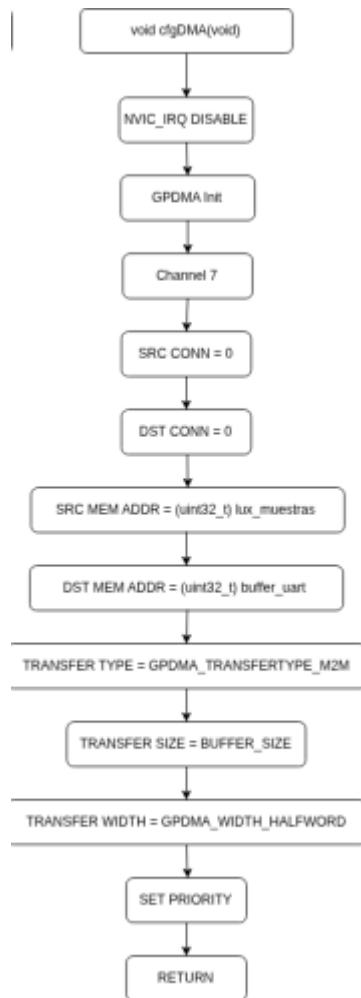
    return;
}

```

4.2. FIRMWARE DEL SSE DE COMUNICACIÓN DE DATOS







```

void cfgGPIO(void){
    PINSEL_CFG_Type cfgTXD2;
    PINSEL_CFG_Type cfgRXD2;

    cfgTXD2.Portnum = PINSEL_PORT_0;
    cfgTXD2.Pinnum = PINSEL_PIN_10;
    cfgTXD2.Funcnum = PINSEL_FUNC_1;
    cfgTXD2.OpenDrain = PINSEL_PINMODE_NORMAL;

    cfgRXD2.Portnum = PINSEL_PORT_0;
    cfgRXD2.Pinnum = PINSEL_PIN_11;
    cfgRXD2.Funcnum = PINSEL_FUNC_1;
    cfgRXD2.OpenDrain = PINSEL_PINMODE_NORMAL;

    PINSEL_ConfigPin(&cfgTXD2);
    PINSEL_ConfigPin(&cfgRXD2);

    return;
}
  
```



```

void cfgUART(void){
    UART_CFG_Type      cfgPinUART2;
    UART_FIFO_CFG_Type  cfgUART2FIFO;

    UART_ConfigStructInit(&cfgPinUART2);
    UART_Init(LPC_UART2, &cfgPinUART2);
    UART_FIFOConfigStructInit(&cfgUART2FIFO);
    UART_FIFOConfig(LPC_UART2, &cfgUART2FIFO);
    UART_IntConfig(LPC_UART2, UART_INTCFG_RBR, ENABLE);
    UART_TxCmd(LPC_UART2, ENABLE);

    NVIC_SetPriority(UART2_IRQn, 1);
    NVIC_EnableIRQ(UART2_IRQn);

    return;
}

```

```

void cfgDMA(void){
    GPDMA_Channel_CFG_Type cfgDMA;

    NVIC_DisableIRQ(DMA_IRQn);
    GPDMA_Init();

    cfgDMA.ChannelNum = 7;
    cfgDMA.SrcConn = 0;
    cfgDMA.DstConn = 0;
    cfgDMA.SrcMemAddr = (uint32_t)lux_muestras;
    cfgDMA.DstMemAddr = (uint32_t)buffer_uart;
    cfgDMA.TransferType = GPDMA_TRANSFERTYPE_M2M;
    cfgDMA.TransferSize = BUFFER_SIZE;
    cfgDMA.TransferWidth = GPDMA_WIDTH_HALFWORD;
    cfgDMA.DMALLI = 0;

    GPDMA_Setup(&cfgDMA);

    NVIC_SetPriority(DMA_IRQn, 0);

    return;
}

```

```

void UART2_IRQHandler(void) {
    static uint8_t received_flag = 0;
    uint8_t receivedData = 0;

    if (UART_GetLineStatus(LPC_UART2) & UART_LSR_RDR) {

        receivedData = UART_ReceiveByte(LPC_UART2);

        if (received_flag == 0) {
            set_point = (uint16_t) (receivedData << 8);
            received_flag = 1;
        } else if (received_flag == 1) {
            set_point = set_point | (uint16_t) receivedData;
            received_flag = 0;
        }

    }

    return;
}

```

```

void sendData(uint16_t lux_muestras) {
    static uint8_t buffer[2] = {0, 0};

    buffer[0] = (uint8_t) (lux_muestras >> 8); //Byte mas alto
    buffer[1] = (uint8_t) (lux_muestras & 0xFF); //Byte mas bajo

    UART_Send(LPC_UART2, buffer, sizeof(buffer), BLOCKING);

    return;
}

```

```

void DMA_IRQHandler(void) {
    if (GPDMA_IntGetStatus(GPDMA_STAT_INT, 7)) {
        if (GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 7)) {
            for (uint8_t i = 0; i < BUFFER_SIZE; i++) {
                uint16_t dato = *((uint16_t *) (BUFFER_1_START) + i);
                sendData(dato);
            }

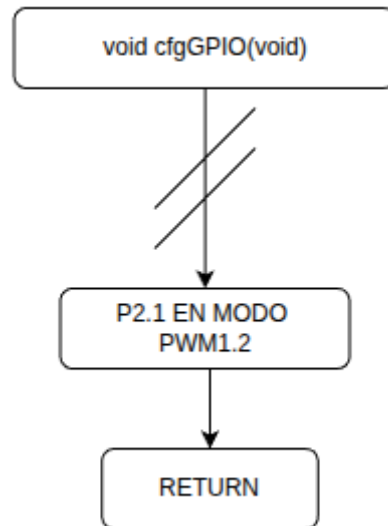
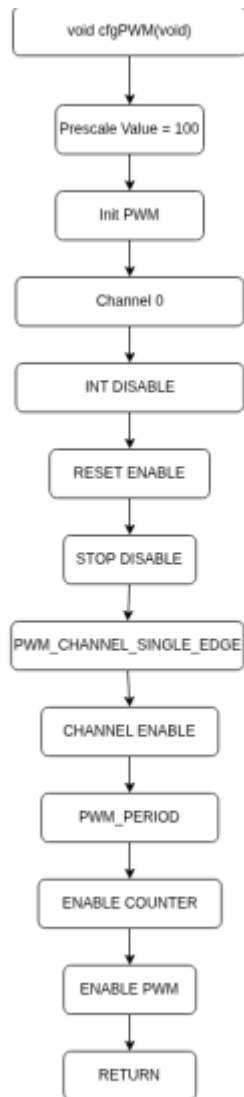
            GPDMA_ClearIntPending(GPDMA_STATCLR_INTTC, 7);
        }

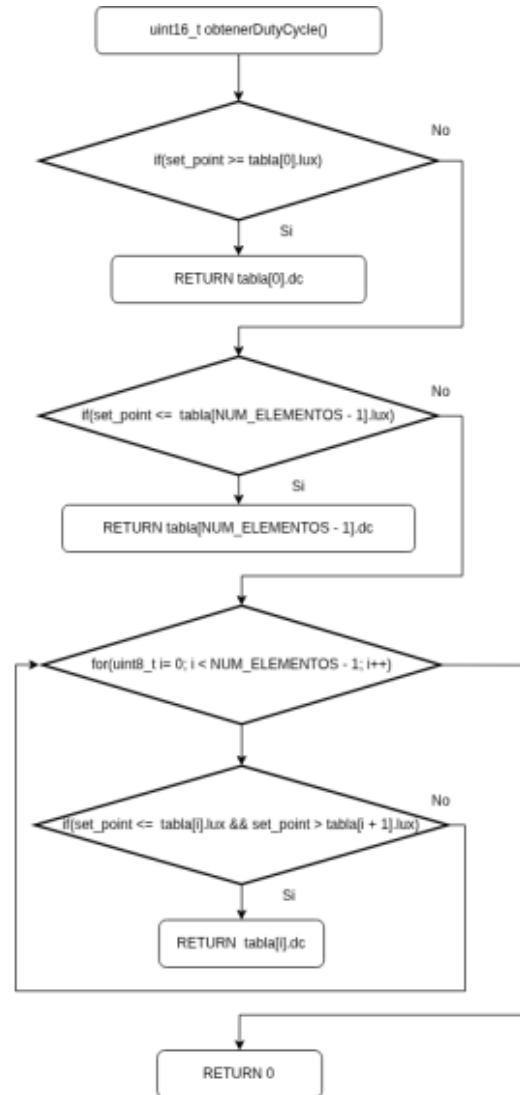
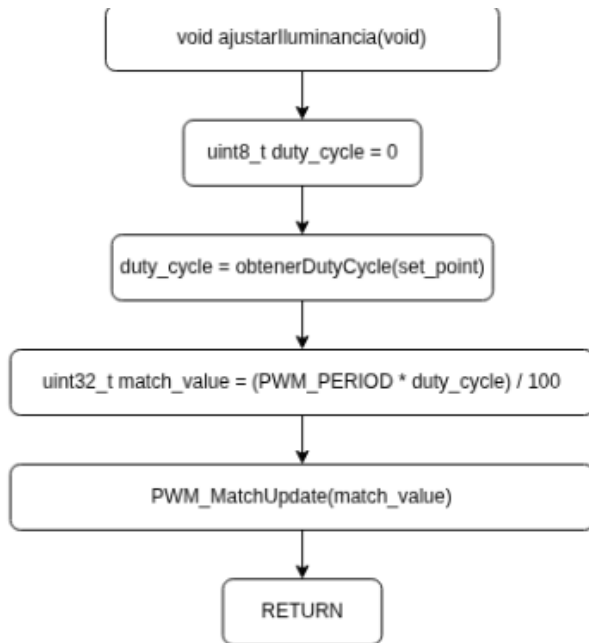
        if (GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 7)) {
            GPDMA_ClearIntPending(GPDMA_STATCLR_INTERR, 7);
        }
    }

    GPDMA_ChannelCmd(7, DISABLE);
    NVIC_DisableIRQ(DMA_IRQn);
    return;
}

```

4.3. FIRMWARE DEL SSE DRIVER / LÁMPARA LED





```

void cfgGPIO(void){
    /* PWM1.2 en P2.1 que no interfiere con displays */
    PINSEL_CFG_Type cfgPinPWM;

    // PWM1.2 en P2.1
    cfgPinPWM.Portnum = PINSEL_PORT_2;
    cfgPinPWM.Pinnum = PINSEL_PIN_1;
    cfgPinPWM.Funcnum = PINSEL_FUNC_1; // PWM1.2
    cfgPinPWM.Pinmode = PINSEL_PINMODE_TRISTATE;
    cfgPinPWM.OpenDrain = PINSEL_PINMODE_NORMAL;

    PINSEL_ConfigPin(&cfgPinPWM);

    return;
}
  
```

```

void cfgPWM(void){
    PWM_TIMERCFG_Type cfgPWMTimer;
    PWM_MATCHCFG_Type cfgPWMMatch;

    // Configurar PWM en modo timer
    cfgPWMTimer.PrescaleOption = PWM_TIMER_PRESCALE_USVAL;
    cfgPWMTimer.PrescaleValue = 100; // 100 us por tick

    PWM_Init(LPC_PWM1, PWM_MODE_TIMER, &cfgPWMTimer);

    // Configurar MR6 como duty cycle (canal PWM1.2)
    cfgPWMMatch.MatchChannel = 2;
    cfgPWMMatch.IntOnMatch = DISABLE;
    cfgPWMMatch.ResetOnMatch = DISABLE;
    cfgPWMMatch.StopOnMatch = DISABLE;

    PWM_ConfigMatch(LPC_PWM1, &cfgPWMMatch);

    PWM_ChannelConfig(LPC_PWM1, 2, PWM_CHANNEL_SINGLE_EDGE);

    PWM_ChannelCmd(LPC_PWM1, 2, ENABLE);

    PWM_MatchUpdate(LPC_PWM1, 0, PWM_PERIOD, PWM_MATCH_UPDATE_NOW);
    PWM_MatchUpdate(LPC_PWM1, 2, 0, PWM_MATCH_UPDATE_NEXT_RST);

    PWM_CounterCmd(LPC_PWM1, ENABLE);
    PWM_ResetCounter(LPC_PWM1);
    PWM_Cmd(LPC_PWM1, ENABLE);

    return;
}

```

```

uint8_t obtenerDutyCycle(void){
    if(set_point >= tabla[0].lux){
        return tabla[0].dc;
    }

    if(set_point <= tabla[NUM_ELEMENTOS - 1].lux){
        return tabla[NUM_ELEMENTOS - 1].dc;
    }

    for (int i = 0; i < NUM_ELEMENTOS - 1; i++){
        if ((set_point <= tabla[i].lux) && (set_point > tabla[i+1].lux) ){
            return tabla[i].dc;
        }
    }

    return 0;
}

```

```

void ajustarIluminancia(void){
    uint8_t duty_cycle = 0;

    duty_cycle = obtenerDutyCycle();

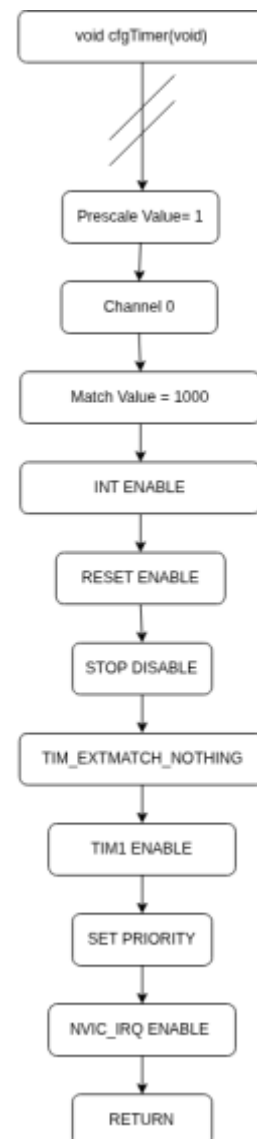
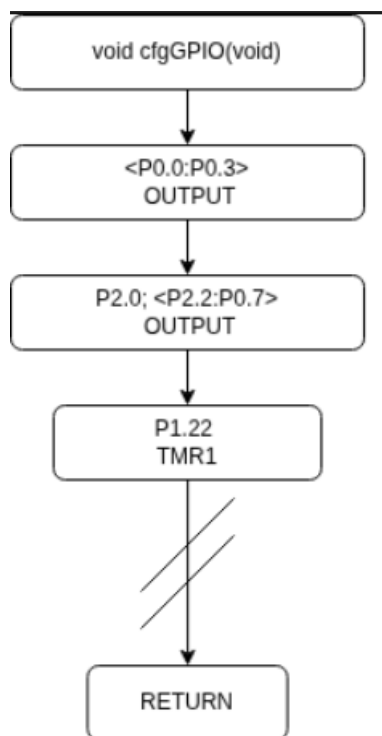
    uint32_t match_value = (PWM_PERIOD * duty_cycle) / 100;

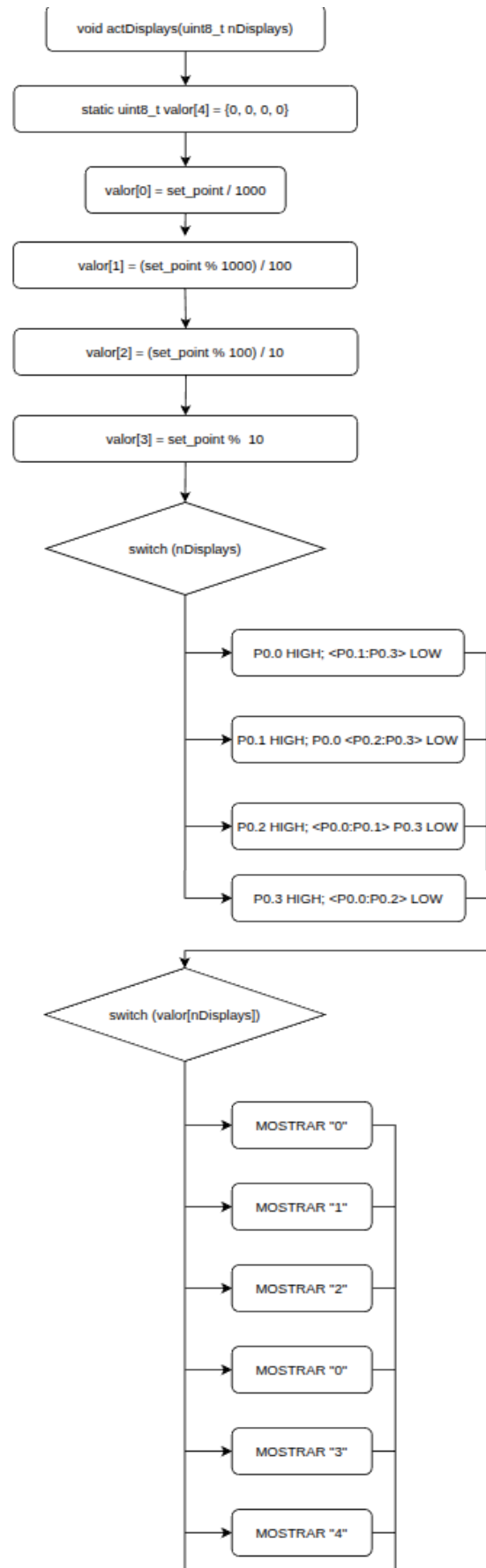
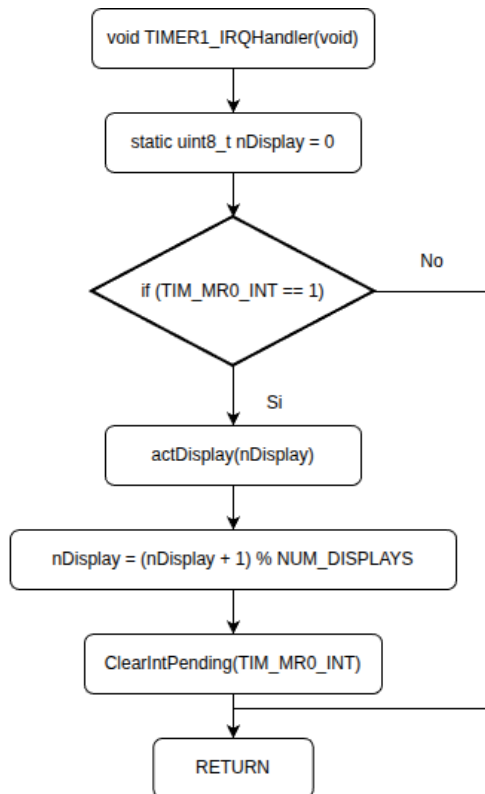
    PWM_MatchUpdate(LPC_PWM1, 2, match_value, PWM_MATCH_UPDATE_NEXT_RST);

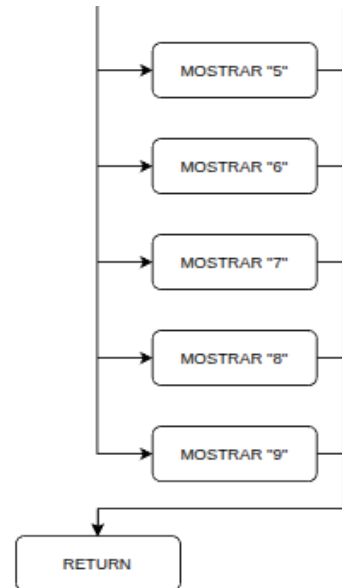
    return;
}

```

4.4. FIRMWARE DEL SSE DE VISUALIZACIÓN DE DATOS







```

void cfgGPIO(void) {
    /* Pines de los display's */
    PINSEL_CFG_Type cfgPinQ; //Transistores de los displays
    PINSEL_CFG_Type cfgPinSeg; //Segmentos

    // P0.0
    cfgPinQ.Portnum = PINSEL_PORT_0;
    cfgPinQ.Pinnum = PINSEL_PIN_0;
    cfgPinQ.Funcnum = PINSEL_FUNC_0;
    cfgPinQ.Pinmode = PINSEL_PINMODE_TRISTATE;
    cfgPinQ.OpenDrain = PINSEL_PINMODE_NORMAL;

    PINSEL_ConfigPin(&cfgPinQ);
    GPIO_SetDir(PORT_0, PIN_0, OUTPUT);

    // P0.1
    cfgPinQ.Pinnum = PINSEL_PIN_1;

    PINSEL_ConfigPin(&cfgPinQ);
    GPIO_SetDir(PORT_0, PIN_1, OUTPUT);

    // P0.2
    cfgPinQ.Pinnum = PINSEL_PIN_2;

    PINSEL_ConfigPin(&cfgPinQ);
    GPIO_SetDir(PORT_0, PIN_2, OUTPUT);

    // P0.3
    cfgPinQ.Pinnum = PINSEL_PIN_3;

    PINSEL_ConfigPin(&cfgPinQ);
    GPIO_SetDir(PORT_0, PIN_3, OUTPUT);

    // P2.0
    cfgPinSeg.Portnum = PINSEL_PORT_2;
    cfgPinSeg.Pinnum = PINSEL_PIN_0;
    cfgPinSeg.Funcnum = PINSEL_FUNC_0;
    cfgPinSeg.Pinmode = PINSEL_PINMODE_TRISTATE;
  
```



```

        cfgPinSeg.OpenDrain = PINSEL_PINMODE_NORMAL;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_0, OUTPUT);

        // P2.7
        cfgPinSeg.Pinnum = PINSEL_PIN_7;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_7, OUTPUT);

        // P2.2
        cfgPinSeg.Pinnum = PINSEL_PIN_2;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_2, OUTPUT);

        // P2.3
        cfgPinSeg.Pinnum = PINSEL_PIN_3;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_3, OUTPUT);

        // P2.4
        cfgPinSeg.Pinnum = PINSEL_PIN_4;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_4, OUTPUT);

        // P2.5
        cfgPinSeg.Pinnum = PINSEL_PIN_5;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_5, OUTPUT);

        // P2.6
        cfgPinSeg.Pinnum = PINSEL_PIN_6;

        PINSEL_ConfigPin(&cfgPinSeg);
        GPIO_SetDir(PORT_2, PIN_6, OUTPUT);

        /* P1.22 como MAT1.0 */
        PINSEL_CFG_Type cfgPinTMR1MAT0;

        cfgPinTMR1MAT0.Portnum = PINSEL_PORT_1;
        cfgPinTMR1MAT0.Pinnum = PINSEL_PIN_22;
        cfgPinTMR1MAT0.Funcnum = PINSEL_FUNC_3;
        cfgPinTMR1MAT0.Pinmode = PINSEL_PINMODE_TRISTATE;
        cfgPinTMR1MAT0.OpenDrain = PINSEL_PINMODE_NORMAL;

        PINSEL_ConfigPin(&cfgPinTMR1MAT0);

        return;
    }

```

```

void cfgTimer(void){
    /* Configuración del TMR1 en modo match - (TMR1 cada 1ms) */
    TIM_TIMERCFG_Type cfgTMR1Mode;
    TIM_MATCHCFG_Type cfgTMR1MAT0;

    // TMR1
    cfgTMR1Mode.PrescaleOption = TIM_PRESCALE_USVAL;
    cfgTMR1Mode.PrescaleValue = 1; // 1 us

    // MATCH 0
    cfgTMR1MAT0.MatchChannel = 0;
    cfgTMR1MAT0.MatchValue = 1000; // 1ms
    cfgTMR1MAT0.IntOnMatch = ENABLE;
    cfgTMR1MAT0.ResetOnMatch = ENABLE;
    cfgTMR1MAT0.StopOnMatch = DISABLE;
    cfgTMR1MAT0.ExtMatchOutputType = TIM_EXTMATCH_NOTHING;

    TIM_Init(LPC_TIM1, TIM_TIMER_MODE, &cfgTMR1Mode);
    TIM_ConfigMatch(LPC_TIM1, &cfgTMR1MAT0);
    TIM_Cmd(LPC_TIM1, ENABLE);

    NVIC_SetPriority(TIMER1_IRQn, 0);
    NVIC_EnableIRQ(TIMER1_IRQn);

    return;
}

```

```

void TIMER1_IRQHandler(void){
    static uint8_t nDisplay = 0;

    if(TIM_GetIntStatus(LPC_TIM1, TIM_MR0_INT)){
        actDisplays(nDisplay);
        nDisplay = (nDisplay + 1) % NUM_DISPLAYS;

        TIM_ClearIntPending(LPC_TIM1, TIM_MR0_INT);
    }

    return;
}

```

```

void actDisplays(uint8_t nDisplays){
    static uint8_t valor[4] = {0, 0, 0, 0};           // mil, cen, dec, uni

    valor[0] = set_point / 1000;
    valor[1] = (set_point % 1000) / 100;
    valor[2] = (set_point % 100) / 10;
    valor[3] = set_point % 10;

    switch(nDisplays){
        case 0:
            GPIO_SetValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_3);
            break;

        case 1:
            GPIO_SetValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_3);
            break;

        case 2:
            GPIO_SetValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_3);
            break;

        case 3:
            GPIO_SetValue(PORT_0, PIN_3);
            GPIO_ClearValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_2);
            break;

        default:
            break;
    }

    switch(valor[nDisplays]){
        case 0:
            GPIO_SetValue(PORT_2, PIN_0);
            GPIO_SetValue(PORT_2, PIN_7);
            GPIO_SetValue(PORT_2, PIN_2);
            GPIO_SetValue(PORT_2, PIN_3);
            GPIO_SetValue(PORT_2, PIN_4);
            GPIO_SetValue(PORT_2, PIN_5);
            GPIO_ClearValue(PORT_2, PIN_6);
            break;

        case 1:
            GPIO_ClearValue(PORT_2, PIN_0);
            GPIO_SetValue(PORT_2, PIN_7);
            GPIO_SetValue(PORT_2, PIN_2);
            GPIO_ClearValue(PORT_2, PIN_3);
            GPIO_ClearValue(PORT_2, PIN_4);
            GPIO_ClearValue(PORT_2, PIN_5);
            GPIO_ClearValue(PORT_2, PIN_6);
            break;

        case 2:
            GPIO_SetValue(PORT_2, PIN_0);

```

```

        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_ClearValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 3:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 4:
        GPIO_ClearValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 5:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_ClearValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 6:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_ClearValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 7:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_ClearValue(PORT_2, PIN_6);
        break;

    case 8:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);

```

```
        GPIO_SetValue(PORT_2, PIN_6);  
        break;  
    case 9:  
        GPIO_SetValue(PORT_2, PIN_0);  
        GPIO_SetValue(PORT_2, PIN_7);  
        GPIO_SetValue(PORT_2, PIN_2);  
        GPIO_ClearValue(PORT_2, PIN_3);  
        GPIO_ClearValue(PORT_2, PIN_4);  
        GPIO_SetValue(PORT_2, PIN_5);  
        GPIO_SetValue(PORT_2, PIN_6);  
        break;  
    default:  
        break;  
}  
  
return;  
}
```

5. INTERFAZ GRÁFICA

Como se mencionó anteriormente, el sistema cuenta con una interfaz gráfica implementada en la PC que permite definir el valor de iluminancia deseado y visualizar, en forma de gráfico, tanto dicho valor como el nivel medido por el sistema. En la *Figura 17* se muestra la interfaz desarrollada.

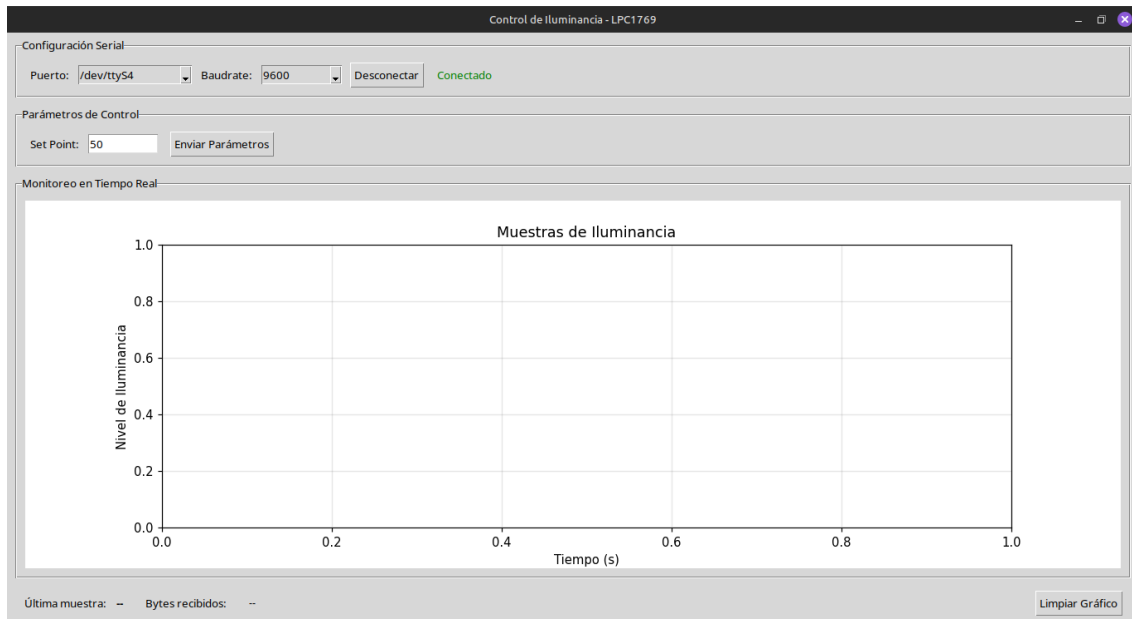


Figura 17: Interfaz gráfica implementada en la PC

En ella se puede apreciar un campo destinado a ingresar el valor de iluminancia deseado [LUX], así como un área donde se visualizan las últimas mediciones registradas.

6. PRUEBAS DE SISTEMA

N° Caso de prueba	Descripción del caso de prueba	Paso	Resultado esperado	Resultado obtenido	Observaciones
1	Prueba del funcionamiento del SSE de visualización de datos	Enviar set point por software	Visualización del valor del set point en los display	PASO	los display al momento de iniciar, se visualiza el número 0 en todos.
2	Prueba de funcionamiento del SSE del OPT101	Tomar muestras desde el ADC del LPC1769	Visualización del valor muestreado por el sensor en los display	PASO	Se utilizaron los display como una forma para comprobar la correcta conversión del valor analogico
3	Prueba de funcionamiento del SSE de Comunicación de Datos	Buffer de muestras del ADC lleno y transferidos correctamente por DMA al periférico	Ver, a través de una interfaz gráfica, un grafico con los valores del ADC	NO PASO	No se visualiza nada en el gráfico, mal configurado el DMA
4	Prueba de funcionamiento del SSE de Comunicación de Datos	Buffer de muestras del ADC lleno y transferido por DMA hacia otro buffer.	Ver, a través de una interfaz gráfica, un gráfico con los valores del ADC	PASO	Se puede visualizar correctamente una gráfica con valores coherentes
5	A través del SSE de Comunicación de Datos, transferir valores en lux	Conversión a lux de los valores almacenados en el buffer luego de ser transferidos por DMA	Visualizar en la gráfica valores en Luz coherentes a lo muestreado por el sensor	NO PASO	Incorrecta conversión de valor binario (valor tomado por el ADC) a Lux.
6	Transmisión UART	Establecer Set Point a utilizar a traves de la interfaz grafica	Se debe visualizar en los display, el valor de set point establecido por el usuario	NO PASO	Mal establecida las prioridades de las interrupciones
7	Transmisión UART	Establecer Set Point a utilizar a traves de la interfaz	Se debe visualizar en los display, el	PASO	Correccion de prioridades, se puede visualizar

		grafica	valor de set point establecido por el usuario adquiridos		correctamente el set point en los display.
8	Funcionamiento del SSE de Lampara Led	Establecido el set point, ajusta la intensidad con un PWM	Aumento o disminución de iluminancia	NO PASO	Lampara Led y Display dejaron de funcionar, problema en la prioridad de las interrupciones y mal uso del PWM
9	Funcionamiento del SSE de Lampara Led	Establecido el set point, ajusta la intensidad con un PWM	Aumento o disminución de iluminancia	NO PASO	corrección de PWM y prioridades, no funciona como lo esperado por utilizar una baja frecuencia y mal usada lógica negativ.
10	Funcionamiento del SSE de Lámpara Led	Establecido el set point, ajusta la intensidad con un PWM	Aumento o disminución de iluminancia	PASO	Corrección de frecuencia, bien usada la lógica negativa.
11	Funcionamiento del sistema de control de iluminacia	Energizar placa	Funcionamiento correcto de todos los SSE en conjuntos	NO PASO	Problema al tratar de controlar la iluminancia medida por el sensor con la lámpara LED

7. CONCLUSIONES

El desarrollo del Sistema de Control de Iluminancia en un recinto permitió integrar de forma práctica los contenidos abordados en la materia. Se logró diseñar y construir un sistema capaz de medir la iluminancia mediante el sensor OPT101, procesar dicha información utilizando la placa LPC1769 y actuar sobre la lámpara LED mediante una señal PWM, con el fin de mantener el nivel definido por el usuario.

El único inconveniente que no pudo resolverse fue que la lámpara no consiguió regular la iluminancia medida por el sensor. Esto ocurre porque el sensor realiza una medición puntual y no es capaz de detectar los valores intermedios generados por la modulación PWM, algo que sí puede registrar un sensor externo.

Para la construcción del proyecto se adoptó un enfoque de desarrollo modular. Cada Subsistema Electrónico (SSE) fue programado y verificado de manera independiente. Una vez comprobado el correcto funcionamiento individual de cada módulo, se integraron todos en el Sistema Electrónico Programable completo. Este método facilitó la detección y corrección de errores, permitió aislar fallas y garantizó una etapa de integración final más ordenada y eficiente.

Además, se desarrolló una interfaz gráfica en Python utilizando diversas librerías especializadas, lo que permitió visualizar las mediciones en tiempo real y ajustar el set-point de forma intuitiva.

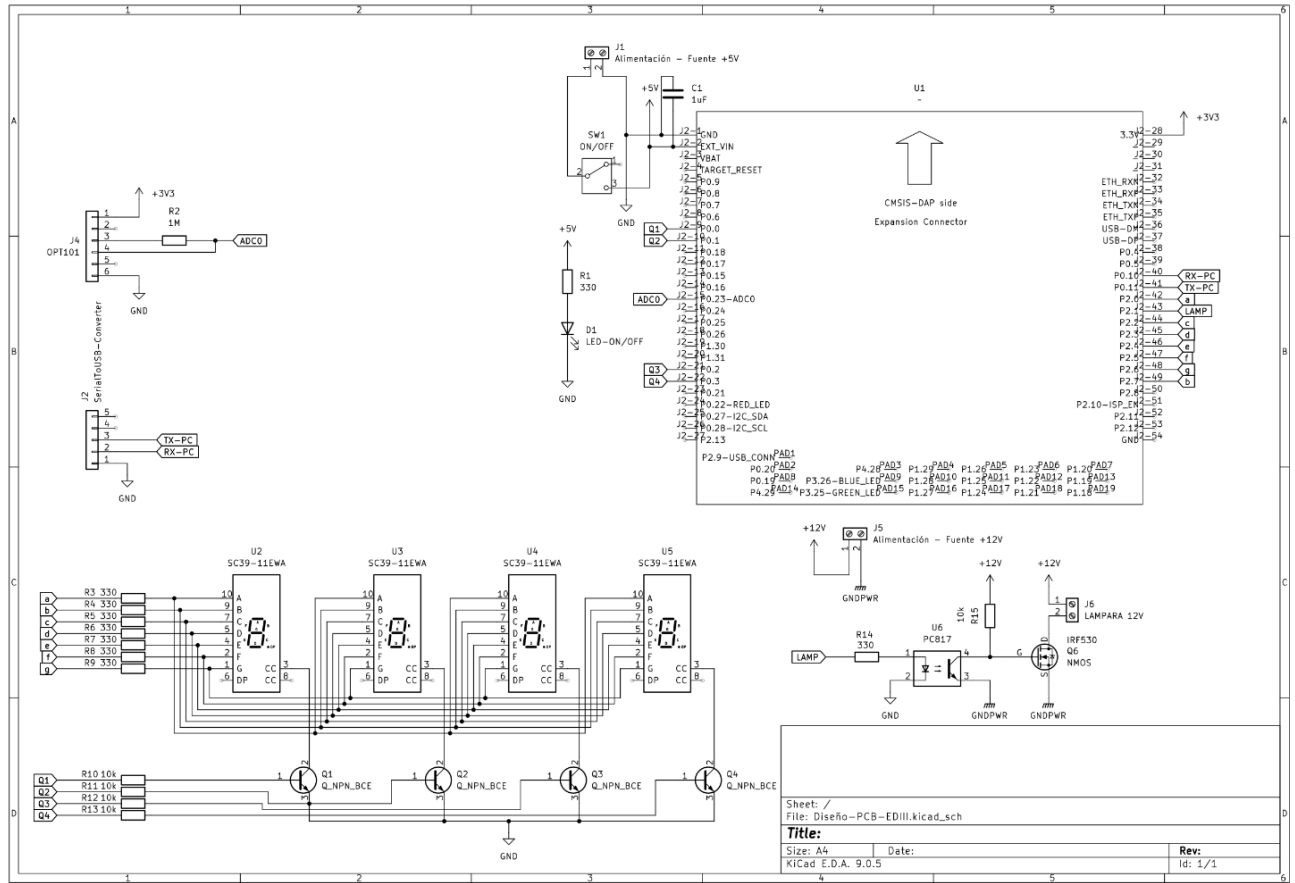
En conclusión, el sistema cumplió con la mayoría de los requisitos planteados, con excepción del mencionado anteriormente. Como propuesta de mejora, se sugiere implementar un esquema de adquisición basado en múltiples muestras consecutivas y el cálculo de su valor promedio, de modo que dicho promedio se utilice como valor efectivo de iluminancia medida.

8. BIBLIOGRAFÍA Y REFERENCIAS

- Hoja de datos de los componentes utilizados.
- Material de las clases.
- Manual de usuario LPC1769.
- Hoja de datos LPC1769.
- Tutorial manejo del sensor OPT101 :
<https://docs.cirkitdesigner.com/component/fbbba498-5a3a-467d-8405-4735bc62cdd4/opt101-light-sensor>
- Documentación de Python:
<https://docs.python.org/3/>

9. ANEXO

8.1 CIRCUITO ESQUEMÁTICO DEL SEP



8.2 FIRMWARE DEL SEP

```
#include "lpc17xx.h"
#include "lpc17xx_pinsel.h"
#include "lpc17xx_nvic.h"
#include "lpc17xx_gpio.h"
#include "lpc17xx_timer.h"
#include "lpc17xx_uart.h"
#include "lpc17xx_adc.h"
#include "lpc17xx_gpdma.h"
#include "lpc17xx_pwm.h"

#define INPUT      (uint8_t) 0
#define OUTPUT     (uint8_t) 1
#define PORT_0     (uint8_t) 0
#define PORT_2     (uint8_t) 2
#define PORT_3     (uint8_t) 3
#define PIN_0      ((uint32_t) (1<<0))
#define PIN_1      ((uint32_t) (1<<1))
#define PIN_2      ((uint32_t) (1<<2))
#define PIN_3      ((uint32_t) (1<<3))
#define PIN_4      ((uint32_t) (1<<4))
#define PIN_5      ((uint32_t) (1<<5))
#define PIN_6      ((uint32_t) (1<<6))
#define PIN_7      ((uint32_t) (1<<7))
#define PIN_22     ((uint32_t) (1<<22))
#define PIN_26     ((uint32_t) (1<<26))

#define NUM_DISPLAYS      (uint8_t)      4
#define VREF               (float) 3.3
#define MAX_VALOR (uint16_t) 4095
#define NUM_ELEMENTOS     26

#define BUFFER_1_START     0x2007C000
#define BUFFER_2_START     0x2007C014
#define BUFFER_SIZE        10

#define PWM_PERIOD         (uint16_t) 100 // Periodo del PWM en ticks (1 KHz con prescaler de 100us)

void cfgGPIO(void);
void cfgTimer(void);
void cfgUART(void);
void cfgADC(void);
void cfgDMA(void);
void cfgPWM(void);
void sendData(uint16_t lux_muestras);
uint16_t calculoLux(uint16_t adc_muestra);
void actDisplays(uint8_t nDisplays);
uint8_t obtenerDutyCycle(void);
void ajustarIluminancia(void);

volatile uint16_t set_point = 0;
volatile uint16_t lux_actual = 0;
volatile uint16_t *lux_muestras = (uint16_t *)BUFFER_1_START;
volatile uint16_t *buffer_uart = (uint16_t *)BUFFER_2_START;
```

```

typedef struct{
    float vout;
    uint16_t lux;
    uint8_t dc;
} Elemento;

static const Elemento tabla[] = {
    {2.68, 295, 0},
    {2.58, 286, 5},
    {2.48, 258, 20},
    {2.38, 231, 35},
    {2.28, 203, 38},
    {2.18, 181, 42},
    {2.08, 168, 45},
    {1.98, 156, 48},
    {1.88, 144, 52},
    {1.78, 134, 55},
    {1.68, 124, 58},
    {1.58, 112, 63},
    {1.48, 99, 67},
    {1.38, 84, 75},
    {1.28, 68, 82},
    {1.18, 53, 84},
    {1.08, 42, 88},
    {0.98, 32, 92},
    {0.88, 19, 96},
    {0.78, 10, 98},
    {0.68, 9, 98},
    {0.58, 8, 98},
    {0.48, 7, 99},
    {0.38, 6, 99},
    {0.28, 5, 100},
    {0.18, 4, 100},
};

int main(void) {

    cfgGPIO();
    cfgTimer();
    cfgPWM();
    cfgADC();
    cfgUART();

    while (1) {}

    return 0;
}

void cfgGPIO(void) {
    /* Pines de los display's */
    PINSEL_CFG_Type cfgPinQ; //Transistores de los displays
    PINSEL_CFG_Type cfgPinSeg; //Segmentos

    // P0.0
    cfgPinQ.Portnum = PINSEL_PORT_0;
    cfgPinQ.Pinnum = PINSEL_PIN_0;
    cfgPinQ.Funcnum = PINSEL_FUNC_0;
    cfgPinQ.Pinmode = PINSEL_PINMODE_TRISTATE;

```

```

cfgPinQ.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinQ);
GPIO_SetDir(PORT_0, PIN_0, OUTPUT);

// P0.1
cfgPinQ.Pinnum = PINSEL_PIN_1;

PINSEL_ConfigPin(&cfgPinQ);
GPIO_SetDir(PORT_0, PIN_1, OUTPUT);

// P0.2
cfgPinQ.Pinnum = PINSEL_PIN_2;

PINSEL_ConfigPin(&cfgPinQ);
GPIO_SetDir(PORT_0, PIN_2, OUTPUT);

// P0.3
cfgPinQ.Pinnum = PINSEL_PIN_3;

PINSEL_ConfigPin(&cfgPinQ);
GPIO_SetDir(PORT_0, PIN_3, OUTPUT);

// P2.0
cfgPinSeg.Portnum = PINSEL_PORT_2;
cfgPinSeg.Pinnum = PINSEL_PIN_0;
cfgPinSeg.Funcnum = PINSEL_FUNC_0;
cfgPinSeg.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinSeg.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_0, OUTPUT);

// P2.7
cfgPinSeg.Pinnum = PINSEL_PIN_7;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_7, OUTPUT);

// P2.2
cfgPinSeg.Pinnum = PINSEL_PIN_2;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_2, OUTPUT);

// P2.3
cfgPinSeg.Pinnum = PINSEL_PIN_3;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_3, OUTPUT);

// P2.4
cfgPinSeg.Pinnum = PINSEL_PIN_4;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_4, OUTPUT);

// P2.5
cfgPinSeg.Pinnum = PINSEL_PIN_5;

```

```

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_5, OUTPUT);

// P2.6
cfgPinSeg.Pinnum = PINSEL_PIN_6;

PINSEL_ConfigPin(&cfgPinSeg);
GPIO_SetDir(PORT_2, PIN_6, OUTPUT);

/* Pines de los Timers*/
PINSEL_CFG_Type cfgPinTMR0MAT1;
PINSEL_CFG_Type cfgPinTMR1MAT0;

/* P1.29 como MAT0.1 */
cfgPinTMR0MAT1.Portnum = PINSEL_PORT_1;
cfgPinTMR0MAT1.Pinnum = PINSEL_PIN_29;
cfgPinTMR0MAT1.Funcnum = PINSEL_FUNC_3;
cfgPinTMR0MAT1.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinTMR0MAT1.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinTMR0MAT1);

/* P1.22 como MAT1.0 */
cfgPinTMR1MAT0.Portnum = PINSEL_PORT_1;
cfgPinTMR1MAT0.Pinnum = PINSEL_PIN_22;
cfgPinTMR1MAT0.Funcnum = PINSEL_FUNC_3;
cfgPinTMR1MAT0.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinTMR1MAT0.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinTMR1MAT0);

/* Pin del Led Testigo */
PINSEL_CFG_Type cfgPinLEDRed;
PINSEL_CFG_Type cfgPinLEDBlue;

// Configuración del LED en P0.22
cfgPinLEDRed.Portnum = PINSEL_PORT_0;
cfgPinLEDRed.Pinnum = PINSEL_PIN_22;
cfgPinLEDRed.Funcnum = PINSEL_FUNC_0;
cfgPinLEDRed.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinLEDRed.OpenDrain = PINSEL_PINMODE_NORMAL;

// Configuración del LED en P3.26
cfgPinLEDBlue.Portnum = PINSEL_PORT_3;
cfgPinLEDBlue.Pinnum = PINSEL_PIN_26;
cfgPinLEDBlue.Funcnum = PINSEL_FUNC_0;
cfgPinLEDBlue.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinLEDBlue.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinLEDRed);
PINSEL_ConfigPin(&cfgPinLEDBlue);

GPIO_SetDir(PORT_0, PIN_22, OUTPUT);
GPIO_SetValue(PORT_0, PIN_22);

GPIO_SetDir(PORT_3, PIN_26, OUTPUT);
GPIO_SetValue(PORT_3, PIN_26);

```

```

/* Pines UART2 */
PINSEL_CFG_Type cfgTXD2;
PINSEL_CFG_Type cfgRXD2;

cfgTXD2.Portnum = PINSEL_PORT_0;
cfgTXD2.Pinnum = PINSEL_PIN_10;
cfgTXD2.Funcnum = PINSEL_FUNC_1;
cfgTXD2.OpenDrain = PINSEL_PINMODE_NORMAL;

cfgRXD2.Portnum = PINSEL_PORT_0;
cfgRXD2.Pinnum = PINSEL_PIN_11;
cfgRXD2.Funcnum = PINSEL_FUNC_1;
cfgRXD2.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgTXD2);
PINSEL_ConfigPin(&cfgRXD2);

/* Pin del ADC */
PINSEL_CFG_Type cfgPinADC;

// Configuración del pin P0.23 ADC Channel 0
cfgPinADC.Portnum = PINSEL_PORT_0;
cfgPinADC.Pinnum = PINSEL_PIN_23;
cfgPinADC.Funcnum = PINSEL_FUNC_1;
cfgPinADC.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinADC.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinADC);

/* PWM1.2 en P2.1 que no interfiere con displays */
PINSEL_CFG_Type cfgPinPWM;

// PWM1.2 en P2.1
cfgPinPWM.Portnum = PINSEL_PORT_2;
cfgPinPWM.Pinnum = PINSEL_PIN_1;
cfgPinPWM.Funcnum = PINSEL_FUNC_1; // PWM1.2
cfgPinPWM.Pinmode = PINSEL_PINMODE_TRISTATE;
cfgPinPWM.OpenDrain = PINSEL_PINMODE_NORMAL;

PINSEL_ConfigPin(&cfgPinPWM);

return;
}

void cfgTimer(void){
    /* Configuración del TMR0 en modo match1 - (TMR0MR1 cada 1s) */
    TIM_TIMERCFG_Type cfgTMR0Mode;
    TIM_MATCHCFG_Type cfgTMR0MAT1;

    // TMR0
    cfgTMR0Mode.PrescaleOption = TIM_PRESCALE_USVAL;
    cfgTMR0Mode.PrescaleValue = 1000; // 1 ms

    // MATCH 1
    cfgTMR0MAT1.MatchChannel = 1;
    cfgTMR0MAT1.MatchValue = 499;
    cfgTMR0MAT1.IntOnMatch = DISABLE;
    cfgTMR0MAT1.ResetOnMatch = ENABLE;
    cfgTMR0MAT1.StopOnMatch = DISABLE;

```



```

    cfgTMR0MAT1.ExtMatchOutputType = TIM_EXTMATCH_TOGGLE;

    TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTMR0Mode);
    TIM_ConfigMatch(LPC_TIM0, &cfgTMR0MAT1);

    /* Configuración del TMR1 en modo match - (TMR1 cada 1ms) */
    TIM_TIMERCFG_Type cfgTMR1Mode;
    TIM_MATCHCFG_Type cfgTMR1MAT0;

    // TMR1
    cfgTMR1Mode.PrescaleOption = TIM_PRESCALE_USVAL;
    cfgTMR1Mode.PrescaleValue = 1; // 1 us

    // MATCH 0
    cfgTMR1MAT0.MatchChannel = 0;
    cfgTMR1MAT0.MatchValue = 1000; // 1ms
    cfgTMR1MAT0.IntOnMatch = ENABLE;
    cfgTMR1MAT0.ResetOnMatch = ENABLE;
    cfgTMR1MAT0.StopOnMatch = DISABLE;
    cfgTMR1MAT0.ExtMatchOutputType = TIM_EXTMATCH_NOHING;

    TIM_Init(LPC_TIM1, TIM_TIMER_MODE, &cfgTMR1Mode);
    TIM_ConfigMatch(LPC_TIM1, &cfgTMR1MAT0);
    TIM_Cmd(LPC_TIM1, ENABLE);

    NVIC_SetPriority(TIMER1_IRQn, 0);
    NVIC_EnableIRQ(TIMER1_IRQn);

    return;
}

void cfgPWM(void){
    PWM_TIMERCFG_Type cfgPWMTimer;
    PWM_MATCHCFG_Type cfgPWMMatch;

    // Configurar PWM en modo timer
    cfgPWMTimer.PrescaleOption = PWM_TIMER_PRESCALE_USVAL;
    cfgPWMTimer.PrescaleValue = 100; // 100 us por tick

    PWM_Init(LPC_PWM1, PWM_MODE_TIMER, &cfgPWMTimer);

    // Configurar MR6 como duty cycle (canal PWM1.2)
    cfgPWMMatch.MatchChannel = 2;
    cfgPWMMatch.IntOnMatch = DISABLE;
    cfgPWMMatch.ResetOnMatch = DISABLE;
    cfgPWMMatch.StopOnMatch = DISABLE;

    PWM_ConfigMatch(LPC_PWM1, &cfgPWMMatch);

    PWM_ChannelConfig(LPC_PWM1, 2, PWM_CHANNEL_SINGLE_EDGE);

    PWM_ChannelCmd(LPC_PWM1, 2, ENABLE);

    PWM_MatchUpdate(LPC_PWM1, 0, PWM_PERIOD, PWM_MATCH_UPDATE_NOW);
    PWM_MatchUpdate(LPC_PWM1, 2, 0, PWM_MATCH_UPDATE_NEXT_RST);

    PWM_CounterCmd(LPC_PWM1, ENABLE);
    PWM_ResetCounter(LPC_PWM1);
    PWM_Cmd(LPC_PWM1, ENABLE);
}

```

```

        return;
    }

void cfgUART(void){
    UART_CFG_Type      cfgPinUART2;
    UART_FIFO_CFG_Type  cfgUART2FIFO;

    UART_ConfigStructInit(&cfgPinUART2);
    UART_Init(LPC_UART2, &cfgPinUART2);
    UART_FIFOConfigStructInit(&cfgUART2FIFO);
    UART_FIFOConfig(LPC_UART2, &cfgUART2FIFO);
    UART_IntConfig(LPC_UART2, UART_INTCFG_RBR, ENABLE);
    UART_TxCmd(LPC_UART2, ENABLE);

    NVIC_SetPriority(UART2_IRQn, 1);
    NVIC_EnableIRQ(UART2_IRQn);

    return;
}

void cfgADC(void){
    ADC_Init(LPC_ADC, 200000);

    ADC_BurstCmd(LPC_ADC, DISABLE);
    ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01);
    ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_RISING);
    ADC_ChannelCmd(LPC_ADC, 0, ENABLE);
    ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);

    NVIC_SetPriority(ADC_IRQn, 2);
    NVIC_EnableIRQ(ADC_IRQn);

    TIM_Cmd(LPC_TIM0, ENABLE);

    return;
}

void cfgDMA(void){
    GPDMA_Channel_CFG_Type cfgDMA;

    NVIC_DisableIRQ(DMA_IRQn);
    GPDMA_Init();

    cfgDMA.ChannelNum = 7;
    cfgDMA.SrcConn = 0;
    cfgDMA.DstConn = 0;
    cfgDMA.SrcMemAddr = (uint32_t)lux_muestras;
    cfgDMA.DstMemAddr = (uint32_t)buffer_uart;
    cfgDMA.TransferType = GPDMA_TRANSFERTYPE_M2M;
    cfgDMA.TransferSize = BUFFER_SIZE;
    cfgDMA.TransferWidth = GPDMA_WIDTH_HALFWORD;
    cfgDMA.DMALLI = 0;

    GPDMA_Setup(&cfgDMA);

    NVIC_SetPriority(DMA_IRQn, 0);

    return;
}

```

```

}

void TIMER1_IRQHandler(void){
    static uint8_t nDisplay = 0;

    if(TIM_GetIntStatus(LPC_TIM1, TIM_MR0_INT)){
        actDisplays(nDisplay);
        nDisplay = (nDisplay + 1) % NUM_DISPLAYS;

        TIM_ClearIntPending(LPC_TIM1, TIM_MR0_INT);
    }

    return;
}

void UART2_IRQHandler(void){
    static uint8_t received_flag = 0;
    uint8_t receivedData = 0;

    if(UART_GetLineStatus(LPC_UART2) & UART_LSR_RDR){

        receivedData = UART_ReceiveByte(LPC_UART2);

        if(received_flag == 0){
            set_point = (uint16_t) (receivedData << 8);
            received_flag = 1;
        } else if(received_flag == 1){
            set_point = set_point | (uint16_t) receivedData;
            received_flag = 0;
        }
    }

    return;
}

void ADC_IRQHandler(void){
    static uint8_t cMuestras = 0;
    uint16_t adc_muestra = 0;
    uint16_t lux_valor = 0;

    while(!(ADC_ChannelGetStatus(LPC_ADC, ADC_CHANNEL_0, ADC_DATA_DONE)));

    adc_muestra = ADC_ChannelGetData(LPC_ADC, ADC_CHANNEL_0);
    lux_valor = calculoLux(adc_muestra);

    *((uint16_t *) (BUFFER_1_START) + cMuestras) = lux_valor;
    GPIO_ClearValue(PORT_0, PIN_22);

    ajustarIluminancia();

    cMuestras++;

    if(cMuestras >= BUFFER_SIZE){
        GPIO_SetValue(PORT_0, PIN_22);

        cfgDMA();

        NVIC_EnableIRQ(DMA_IRQn);
        GPDMA_ChannelCmd(7, ENABLE);
    }
}

```

```

        cMuestras = 0;
    }

    return;
}

void DMA_IRQHandler(void){
    if(GPDMA_IntGetStatus(GPDMA_STAT_INT, 7)){
        if(GPDMA_IntGetStatus(GPDMA_STAT_INTTC, 7)){
            for(uint8_t i = 0; i < BUFFER_SIZE; i++){
                uint16_t dato = *((uint16_t *) (BUFFER_1_START) + i);
                sendData(dato);
            }

            GPDMA_ClearIntPending(GPDMA_STATCLR_INTTC, 7);
        }
        if(GPDMA_IntGetStatus(GPDMA_STAT_INTERR, 7)){
            GPDMA_ClearIntPending(GPDMA_STATCLR_INTERR, 7);
        }
    }

    GPDMA_ChannelCmd(7, DISABLE);
    NVIC_DisableIRQ(DMA_IRQn);
    return;
}

void sendData(uint16_t lux_muestras){
    static uint8_t buffer[2] = {0, 0};

    buffer[0] = (uint8_t)(lux_muestras >> 8); //Byte mas alto
    buffer[1] = (uint8_t)(lux_muestras & 0xFF); //Byte mas bajo

    UART_Send(LPC_UART2, buffer, sizeof(buffer), BLOCKING);

    return;
}

uint16_t calculoLux(uint16_t adc_muestra){
    float vout = 0;
    uint16_t lux_valor = 0;

    vout = (adc_muestra * VREF) / (float)MAX_VALOR;

    if (vout >= tabla[0].vout){
        return tabla[0].lux;
    }

    if (vout <= tabla[NUM_ELEMENTOS - 1].vout){
        return tabla[NUM_ELEMENTOS - 1].lux;
    }

    for (int i = 0; i < NUM_ELEMENTOS - 1; i++){
        if ( (vout <= tabla[i].vout) && (vout > tabla[i+1].vout) ){
            lux_valor = tabla[i].lux;
            return lux_valor;
        }
    }
}

```

```

        return 0;
    }

uint8_t obtenerDutyCycle(void) {

    if(set_point >= tabla[0].lux){
        return tabla[0].dc;
    }

    if(set_point <= tabla[NUM_ELEMENTOS - 1].lux){
        return tabla[NUM_ELEMENTOS - 1].dc;
    }

    for (int i = 0; i < NUM_ELEMENTOS - 1; i++){
        if ((set_point <= tabla[i].lux) && (set_point > tabla[i+1].lux) ){
            return tabla[i].dc;
        }
    }

    return 0;
}

void ajustarIluminancia(void){
    uint8_t duty_cycle = 0;

    duty_cycle = obtenerDutyCycle();

    uint32_t match_value = (PWM_PERIOD * duty_cycle) / 100;

    PWM_MatchUpdate(LPC_PWM1, 2, match_value, PWM_MATCH_UPDATE_NEXT_RST);

    return;
}

void actDisplays(uint8_t nDisplays){
    static uint8_t valor[4] = {0, 0, 0, 0};          // mil, cen, dec, uni

    valor[0] = set_point / 1000;
    valor[1] = (set_point % 1000) / 100;
    valor[2] = (set_point % 100) / 10;
    valor[3] = set_point % 10;

    switch(nDisplays){
        case 0:
            GPIO_SetValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_3);
            break;

        case 1:
            GPIO_SetValue(PORT_0, PIN_1);
            GPIO_ClearValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_3);
            break;

        case 2:
            GPIO_SetValue(PORT_0, PIN_2);
            GPIO_ClearValue(PORT_0, PIN_0);
            GPIO_ClearValue(PORT_0, PIN_1);

```

```

        GPIO_ClearValue(PORT_0, PIN_3);
        break;

    case 3:
        GPIO_SetValue(PORT_0, PIN_3);
        GPIO_ClearValue(PORT_0, PIN_0);
        GPIO_ClearValue(PORT_0, PIN_1);
        GPIO_ClearValue(PORT_0, PIN_2);
        break;

    default:
        break;
}

switch(valor[nDisplays]){
    case 0:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_ClearValue(PORT_2, PIN_6);
        break;

    case 1:
        GPIO_ClearValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_ClearValue(PORT_2, PIN_6);
        break;

    case 2:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_ClearValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 3:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 4:
        GPIO_ClearValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 5:

```

```

        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_ClearValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 6:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_ClearValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 7:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_ClearValue(PORT_2, PIN_5);
        GPIO_ClearValue(PORT_2, PIN_6);
        break;

    case 8:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_SetValue(PORT_2, PIN_3);
        GPIO_SetValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    case 9:
        GPIO_SetValue(PORT_2, PIN_0);
        GPIO_SetValue(PORT_2, PIN_7);
        GPIO_SetValue(PORT_2, PIN_2);
        GPIO_ClearValue(PORT_2, PIN_3);
        GPIO_ClearValue(PORT_2, PIN_4);
        GPIO_SetValue(PORT_2, PIN_5);
        GPIO_SetValue(PORT_2, PIN_6);
        break;

    default:
        break;
}

return;
}

```

8.3 LISTA DE MATERIALES

- 1 x LPC1769
- 1 x PC817
- 9 x Resistencias 330[ohm]
- 5 x Resistencias 10k[ohm]
- 1 x Resistencia 1M[ohm]
- Lampara Led 12v
- 1 x Capacitor 1uF
- 4 x BC547
- Sensor OPT101
- 1 x Led Verde
- 4 x Displays Cátodo Común - Rojos
- 1 x Interruptor
- 1 x IRF530
- Conversor USB - TTL - PL2303
- Cable UTP
- Herramientas (Soldador, Estaño, Pinza de Punta, Alicates, Pelacable, Destornillador)
- Fuente de celular 5v
- Fuente de 12v
- 3 x Borneras de 2 pines
- Pines Macho arduino
- Pines Hembra arduino
- Madera
- Conector Hembra

8.4 HOJAS DE DATOS

- Sensor OPT101:

<https://www.alldatasheet.com/datasheet-pdf/view/545673/TI/OPT101.html>

- LPC1769:

https://www.nxp.com/docs/en/data-sheet/LPC1769_68_67_66_65_64_63.pdf

- PC817:

<https://www.alldatasheet.com/datasheet-pdf/view/43368/SHARP/PC817.html>

- BC547:

<https://www.alldatasheet.com/datasheet-pdf/view/11551/ONSEMI/BC547.html>

- IRF530:

<https://www.alldatasheet.com/datasheet-pdf/pdf/773455/ONSEMI/IRF530.html>